THE UNIVERSITY OF MELBOURNE

COMP90038
Algorithms and Complexity

Lecture 7: Graphs and Graph Concepts

Olya Ohrimenko
(with thanks to Harald Søndergaard)

Flight search

- What information needs to be stored?
- How can it be stored?
- How is it used to find the fastest route/cheapest flight?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications

- Social networks

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications

- Social networks
- Travelling sales person problem

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications



THE UNIVERSITY OF MELBOURNE

- Social networks
- Travelling sales person problem
- Page rank (website ranking)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications

- Social networks
- Travelling sales person problem
- Page rank (website ranking)
- Scheduling

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications



THE UNIVERSITY OF MELBOURNE

- Social networks
- Travelling sales person problem
- Page rank (website ranking)
- Scheduling
- Gene regulatory network

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

(Some) Graph applications

- Social networks
- Travelling sales person problem
- Page rank (website ranking)
- Scheduling
- Gene regulatory network
- Neural networks and LLMs can be seen as very large computational graphs

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs and Trees

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs and Trees

- One instance of the **exhaustive search** paradigm is **graph traversal.**

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs and Trees



THE UNIVERSITY OF MELBOURNE

- One instance of the **exhaustive search** paradigm is **graph traversal.**
- After this lecture we shall look at two ways of systematically visiting every node of a graph, namely **depth-first** and **breadth-first search**.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs and Trees

THE UNIVERSITY OF MELBOURNE

- One instance of the **exhaustive search** paradigm is **graph traversal.**
- After this lecture we shall look at two ways of systematically visiting every node of a graph, namely **depth-first** and **breadth-first search**.
- These two methods of graph traversal form the backbone of a surprisingly large number of useful graph algorithms.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs and Trees

- One instance of the **exhaustive search** paradigm is **graph traversal.**
- After this lecture we shall look at two ways of systematically visiting every node of a graph, namely **depth-first** and **breadth-first search**.
- These two methods of graph traversal form the backbone of a surprisingly large number of useful graph algorithms.
- The graph algorithms are useful because of the large number of practical problems we can model as graph problems, in network design, flow design, planning, scheduling, route finding, and other logistics applications.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

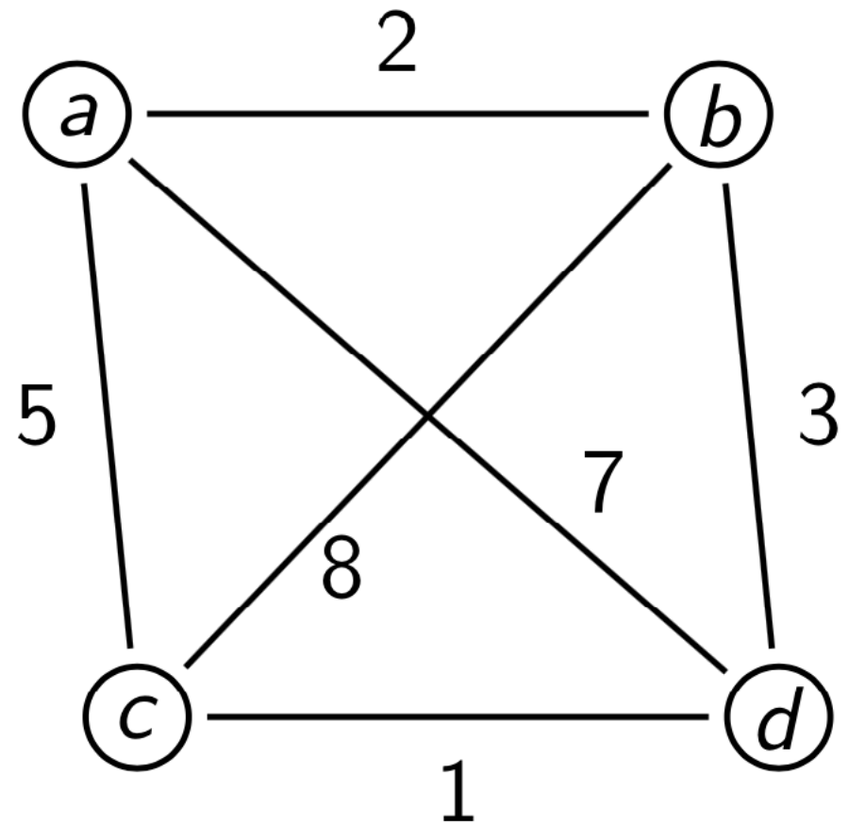
Graphs and Trees

THE UNIVERSITY OF MELBOURNE

- One instance of the **exhaustive search** paradigm is **graph traversal.**
- After this lecture we shall look at two ways of systematically visiting every node of a graph, namely **depth-first** and **breadth-first search**.
- These two methods of graph traversal form the backbone of a surprisingly large number of useful graph algorithms.
- The graph algorithms are useful because of the large number of practical problems we can model as graph problems, in network design, flow design, planning, scheduling, route finding, and other logistics applications.
- Moreover, important numeric and logic problems can be **reduced** to graph problems—more on this in Week 12

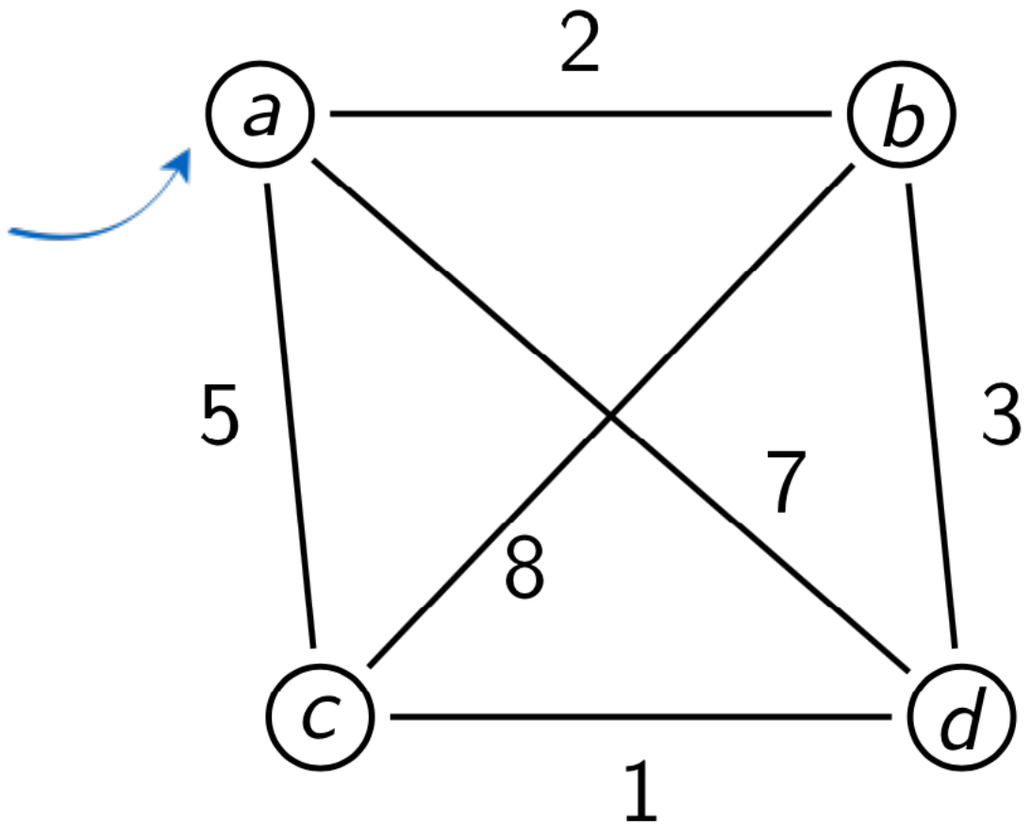
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



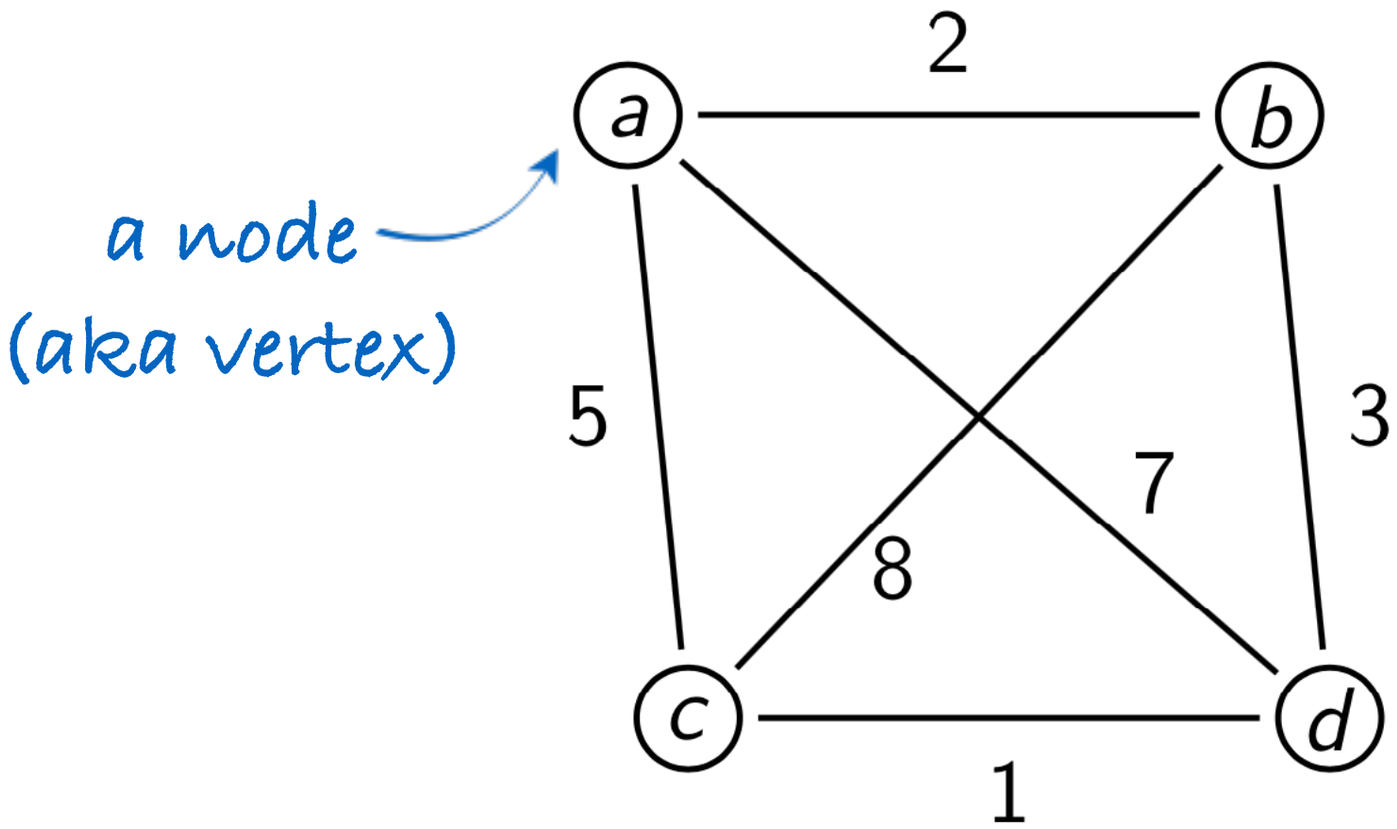
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



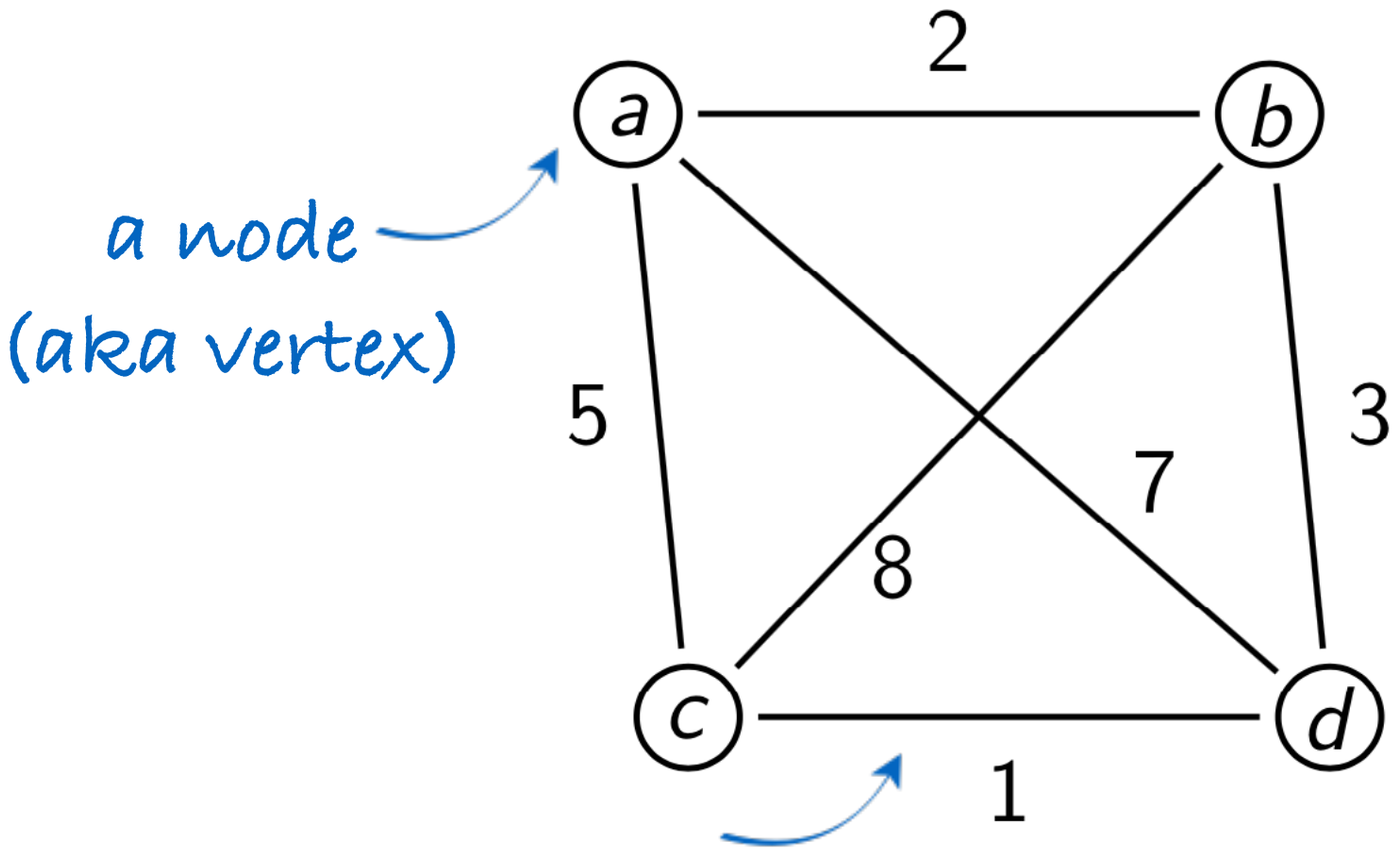
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



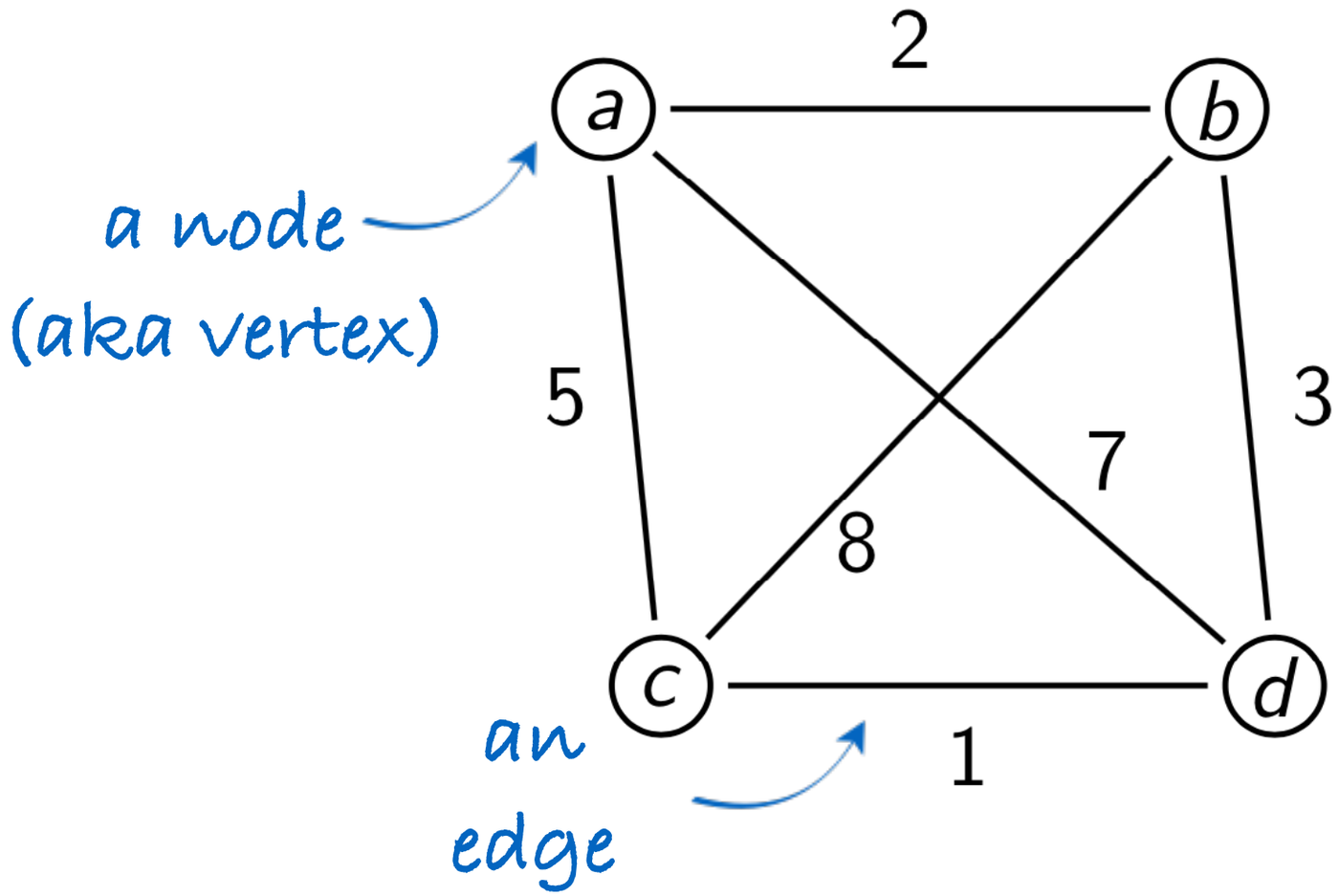
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



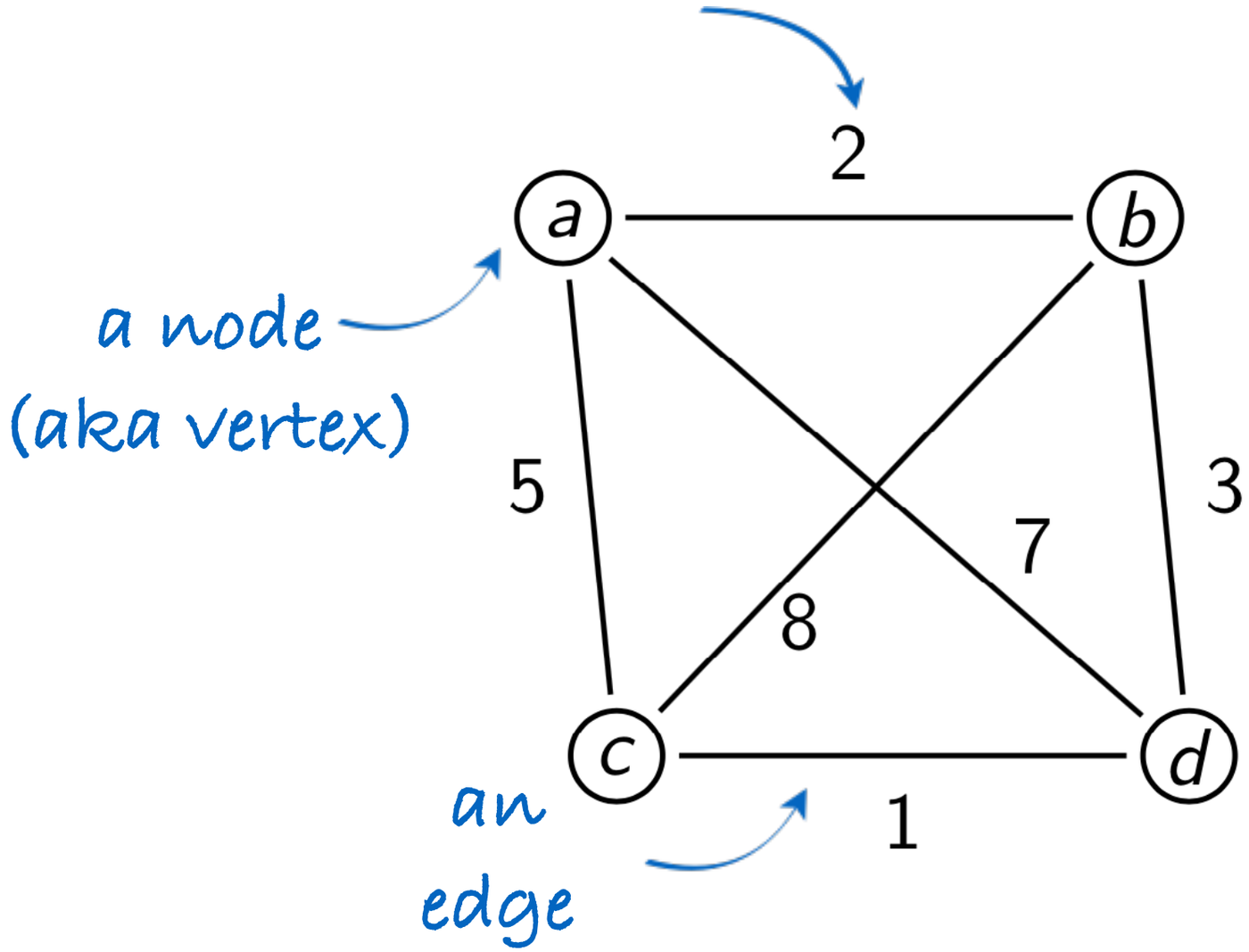
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



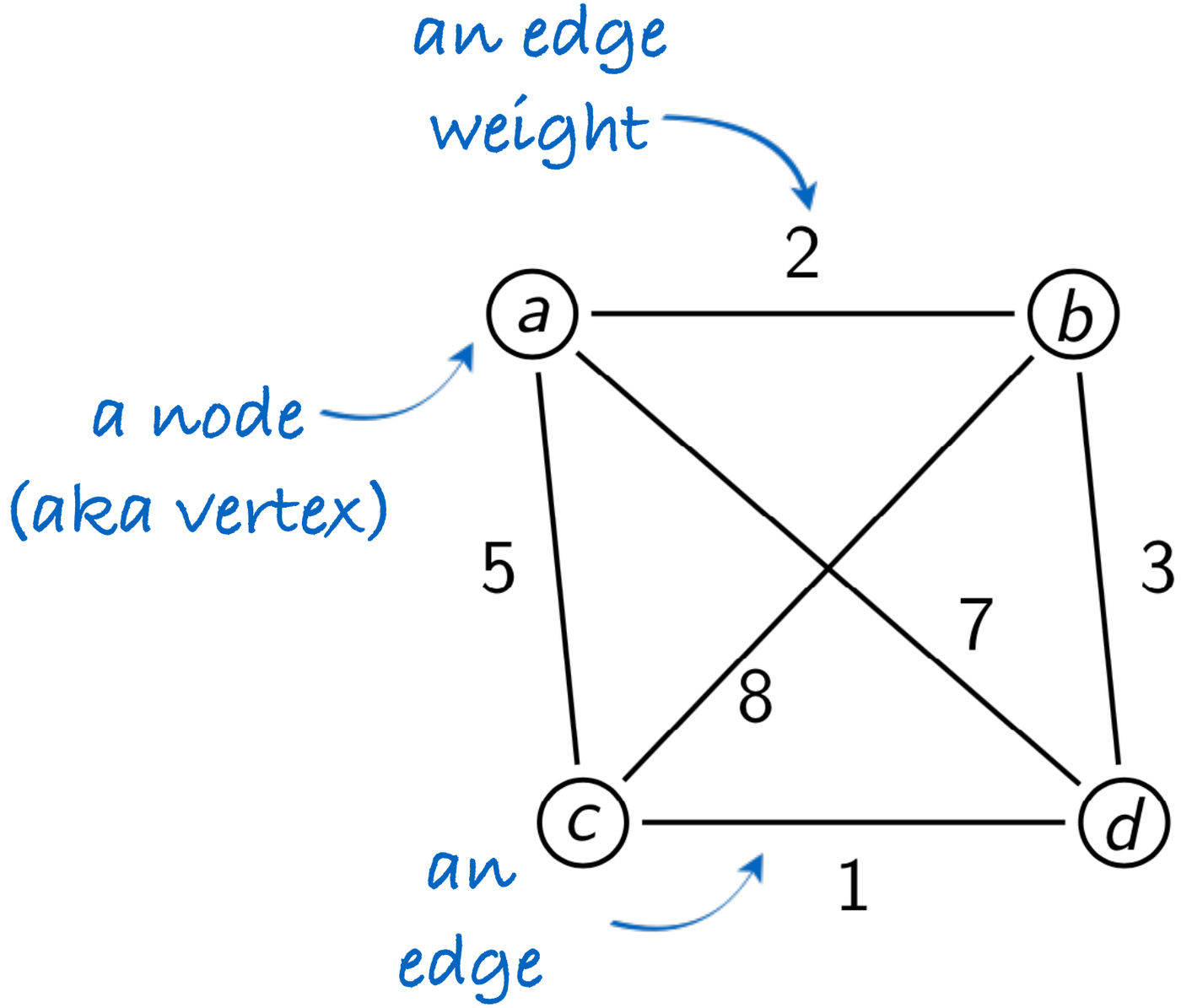
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



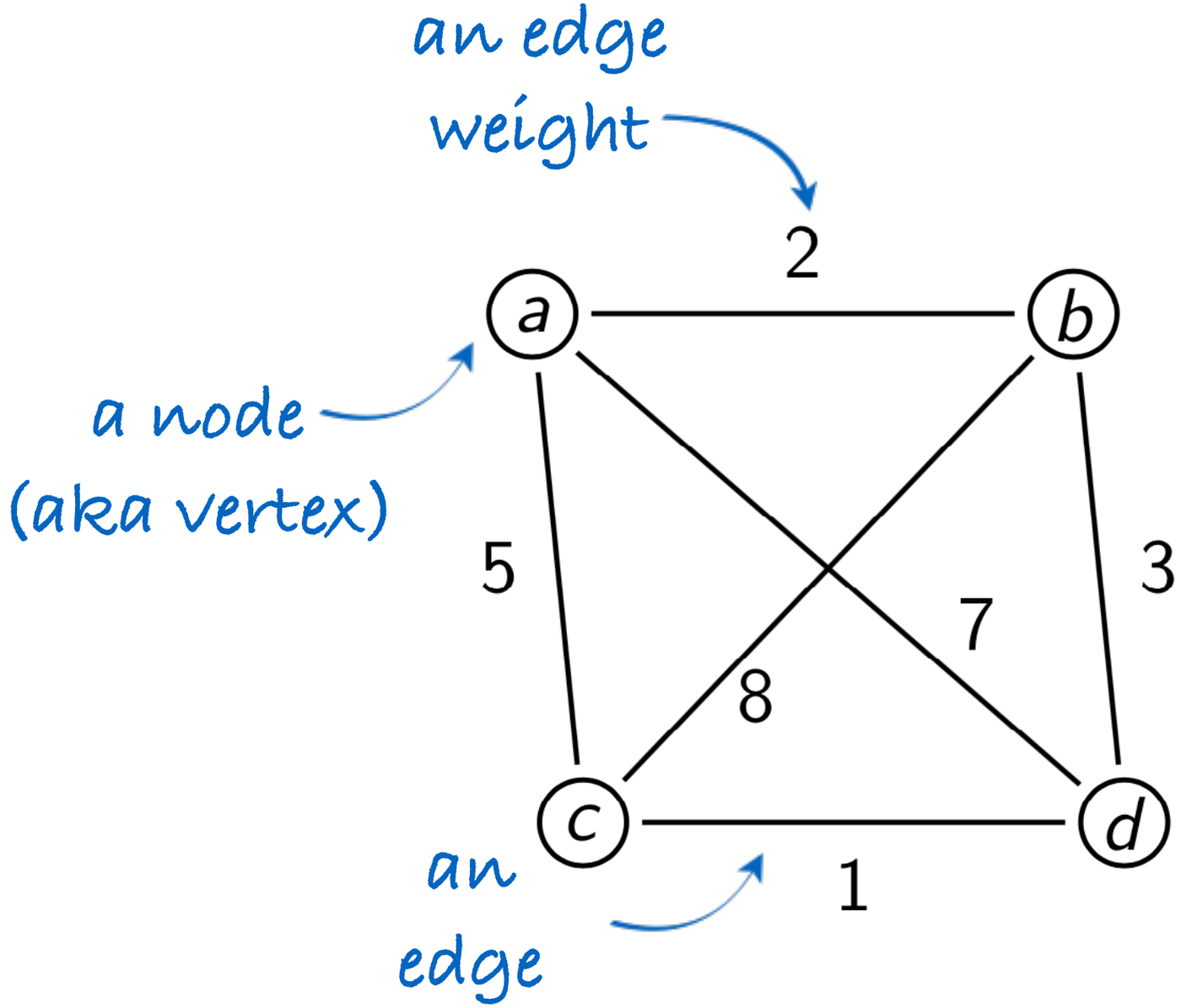
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Basic Graph Concepts



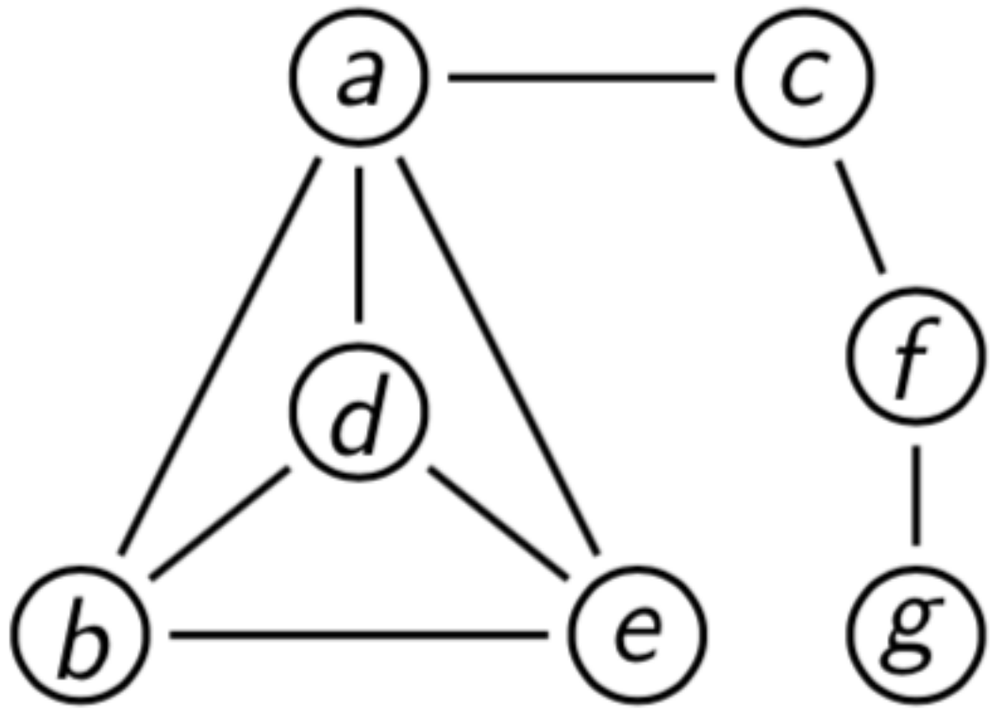
This graph is undirected since edges do not have directions associated with them.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Concepts

THE UNIVERSITY OF MELBOURNE

Undirected Graph



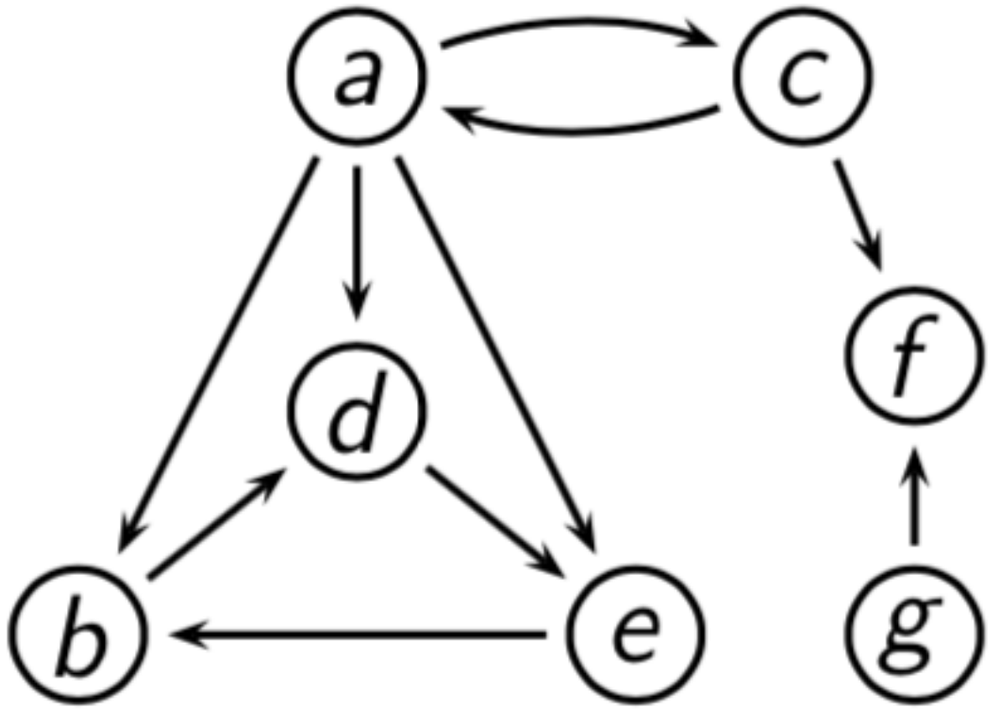
(Edges do not have directions associated with them.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Concepts

THE UNIVERSITY OF MELBOURNE

Directed Graph



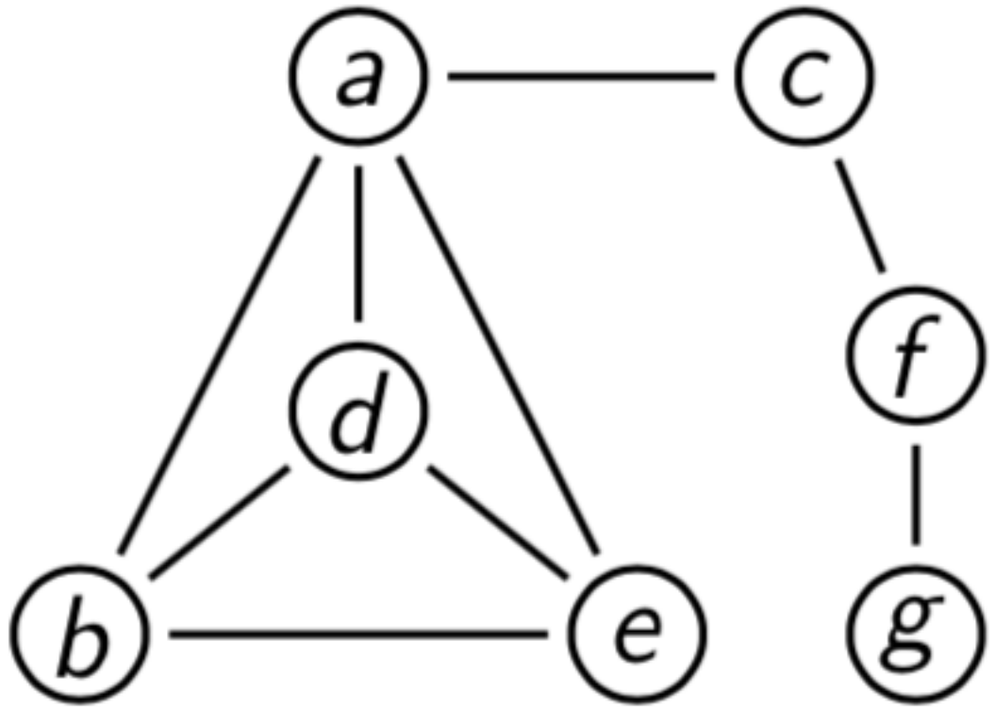
(Each edge has an associated **direction**.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Concepts

THE UNIVERSITY OF MELBOURNE

Connected Graph



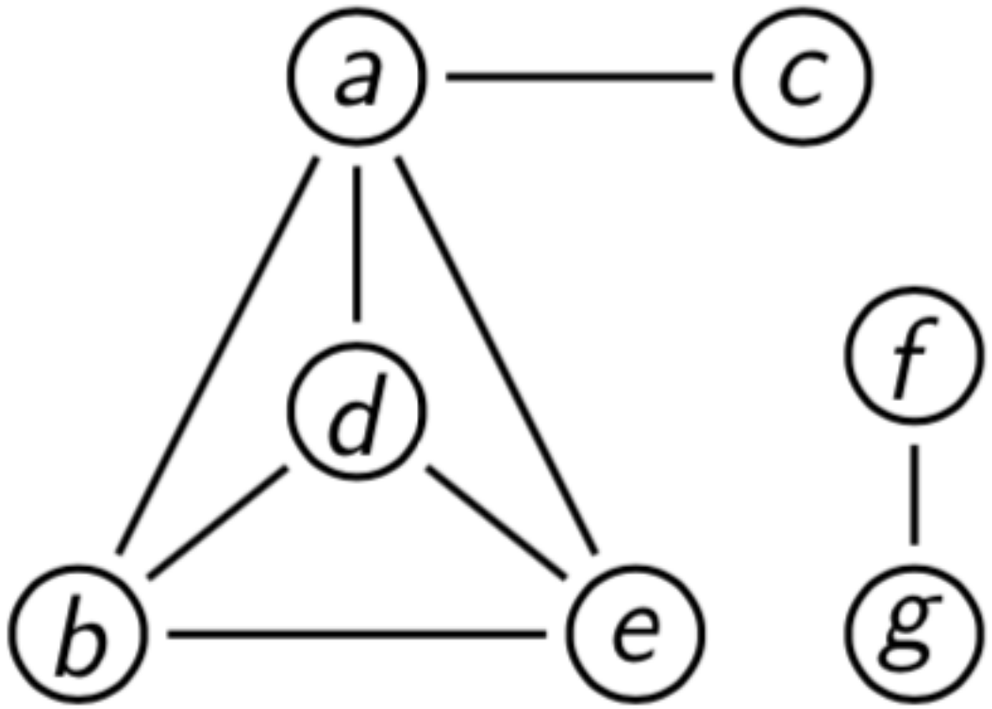
(Each node is reachable from all others by following edges.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Concepts

THE UNIVERSITY OF MELBOURNE

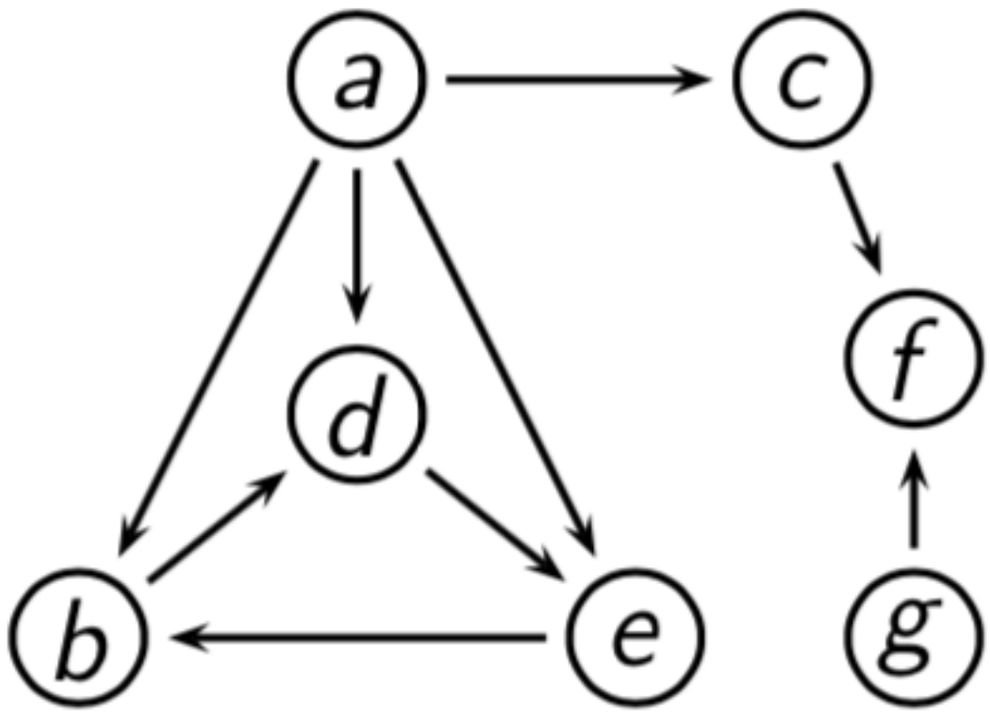
Not Connected Graph, with 2 Components



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs, Mathematically

- A Graph **G** is a pair: ⟨**V**,**E**⟩
  - **V** : set of **nodes** (aka **vertices**)
  - **E** : set of **edges** (a binary relation on **V**)
    - (*u*,*v*) ∈ **E** means there is an edge from *u* to *v*

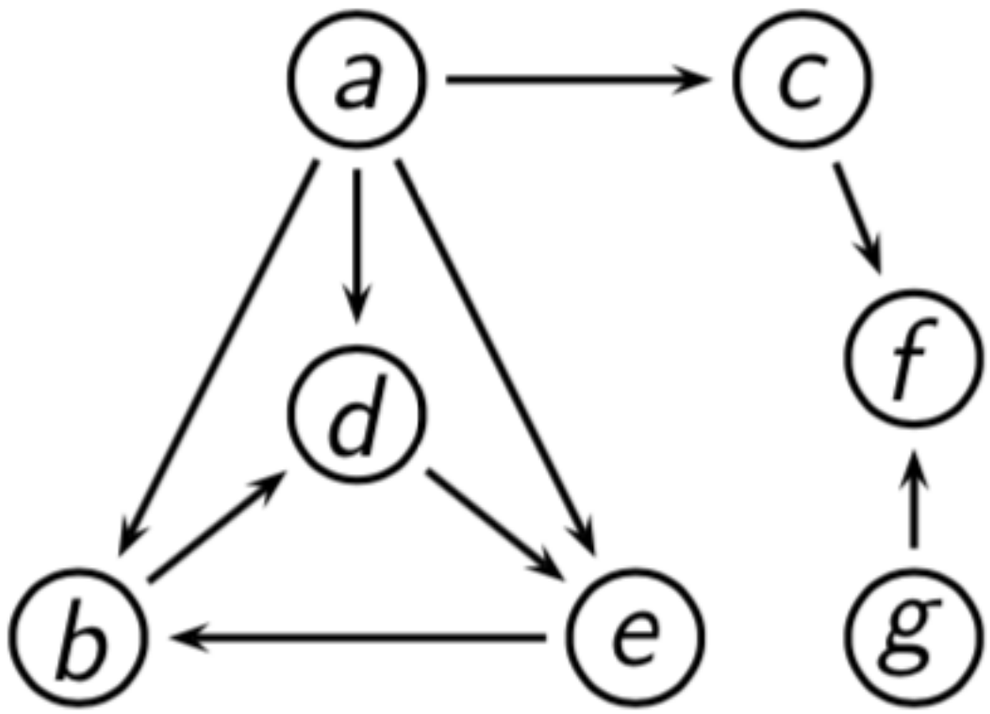


= {(a, b), (a, c), (a, d),
(a, e), (b, d), (b, e),
(c, f), (d, e), (f, g)}

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs, Mathematically

- A Graph **G** is a pair: ⟨**V**,**E**⟩
  - **V** : set of **nodes** (aka **vertices**)
  - **E** : set of **edges** (a binary relation on **V**)
    - (*u*,*v*) ∈ **E** means there is an edge from *u* to *v*



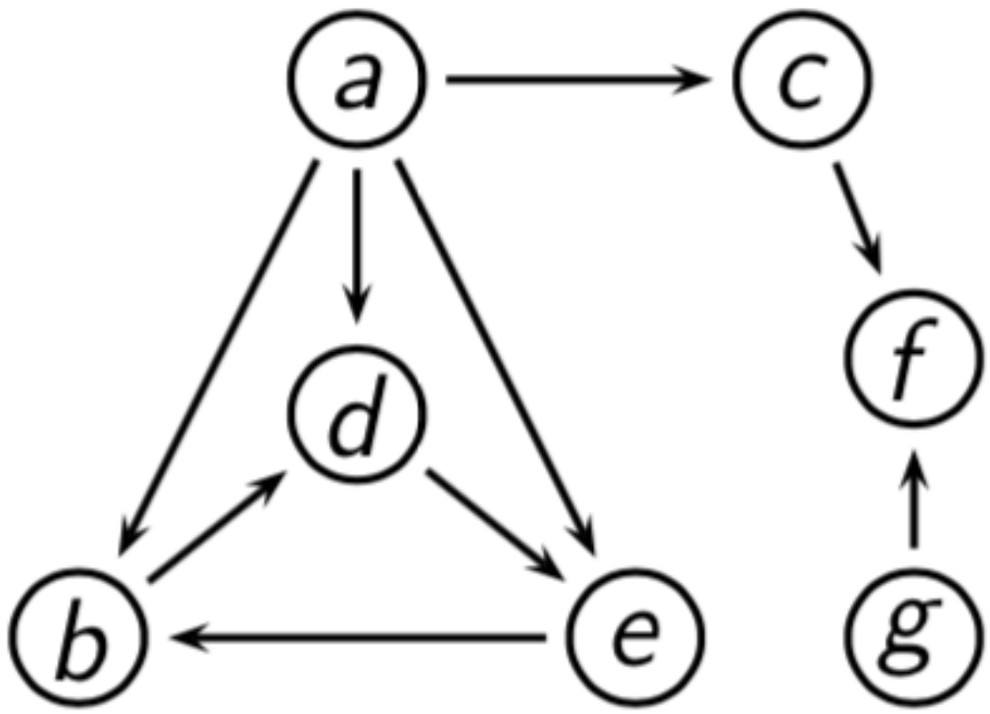
V = {a,b,c,d,e,f,g}

= {(a, b), (a, c), (a, d),
(a, e), (b, d), (b, e),
(c, f), (d, e), (f, g)}

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs, Mathematically

- A Graph **G** is a pair: ⟨**V**,**E**⟩
  - **V** : set of **nodes** (aka **vertices**)
  - **E** : set of **edges** (a binary relation on **V**)
    - (*u*,*v*) ∈ **E** means there is an edge from *u* to *v*



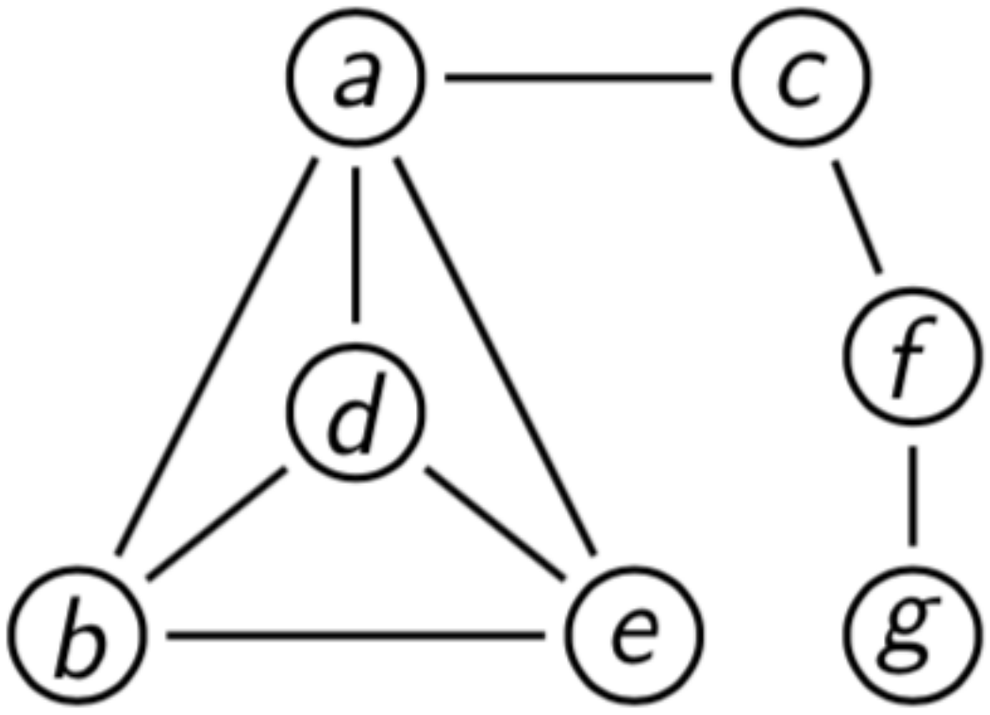
V = {a,b,c,d,e,f,g}

E = {(a, b), (a, c), (a, d), (a, e), (b, d), (c, f), (d, e), (e, b), (g, f)}

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs, Mathematically

- A Graph **G** is a pair: ⟨**V**,**E**⟩
  - **V** : set of **nodes** (aka **vertices**)
  - **E** : set of **edges** (a binary relation on **V**)
    - (*u*,*v*) ∈ **E** means there is an edge from *u* to *v*



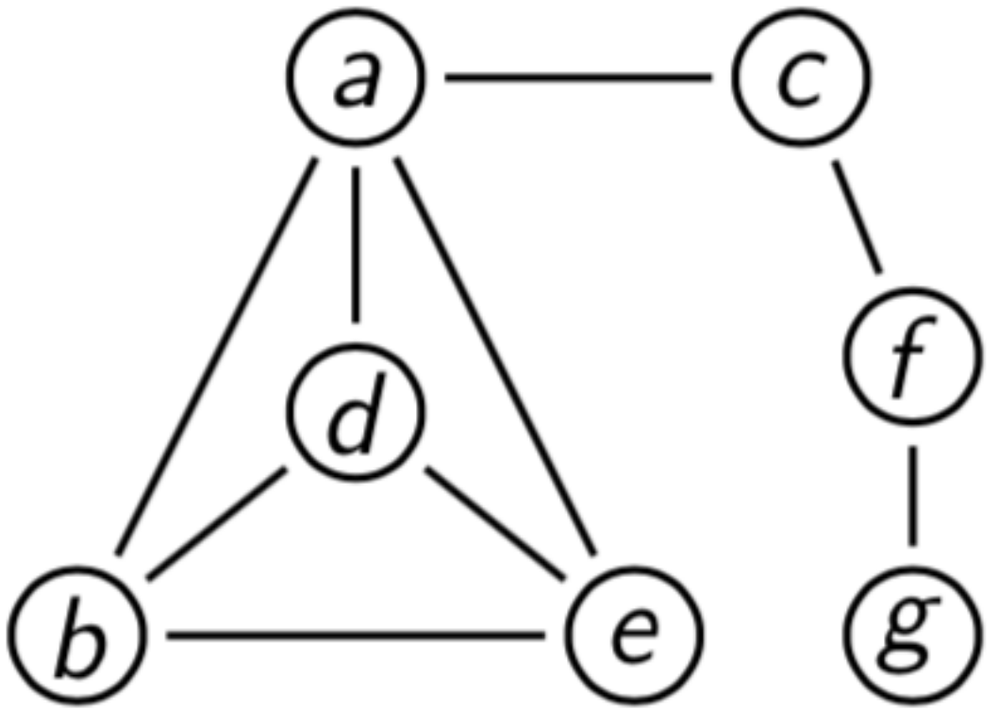
V = {a,b,c,d,e,f,g}

E = {(a, b), (a, c), (a, d),
(a, e), (b, d), (b, e),
(c, f), (d, e), (f, g)}

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graphs, Mathematically

- A Graph **G** is a pair: ⟨**V**,**E**⟩
  - **V** : set of **nodes** (aka **vertices**)
  - **E** : set of **edges** (a binary relation on **V**)
    - (*u*,*v*) ∈ **E** means there is an edge from *u* to *v*



V = {a,b,c,d,e,f,g}

E = **symmetric closure of**
{(a, b), (a, c), (a, d),
(a, e), (b, d), (b, e),
(c, f), (d, e), (f, g)}

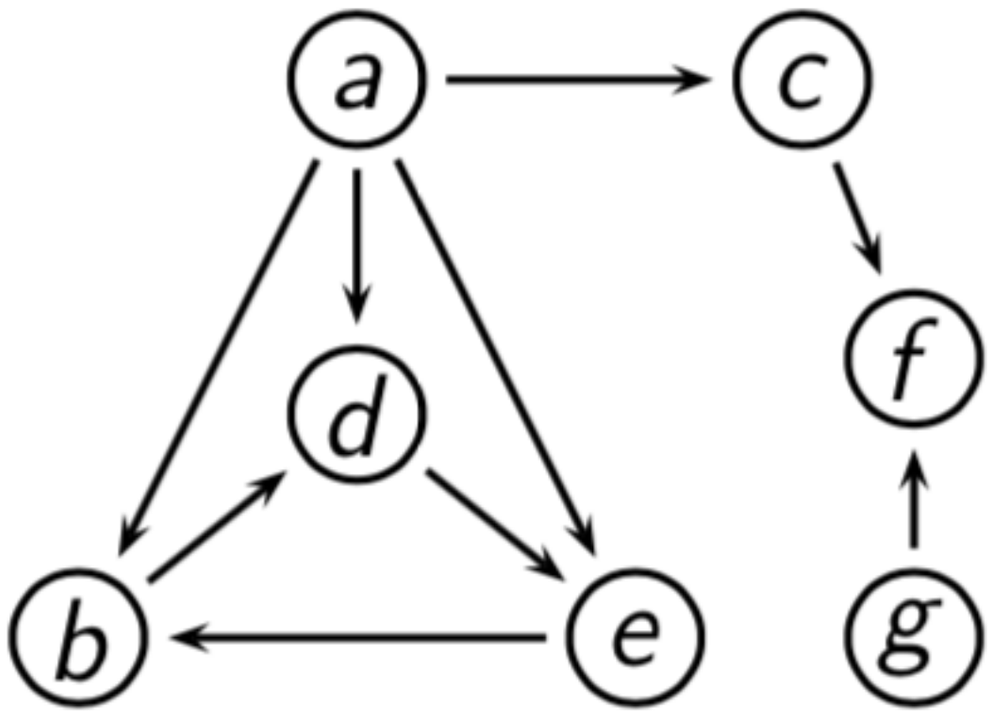
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Degrees of Nodes

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Degrees of Nodes

- If (*v*, *u*) ∈ E then *v* and *u* are **adjacent**, or **neighbours**
- Edge (v, u) is **incident** on, or **connects**, v and u
- **Degree of a node v**: number of edges incident on **v**



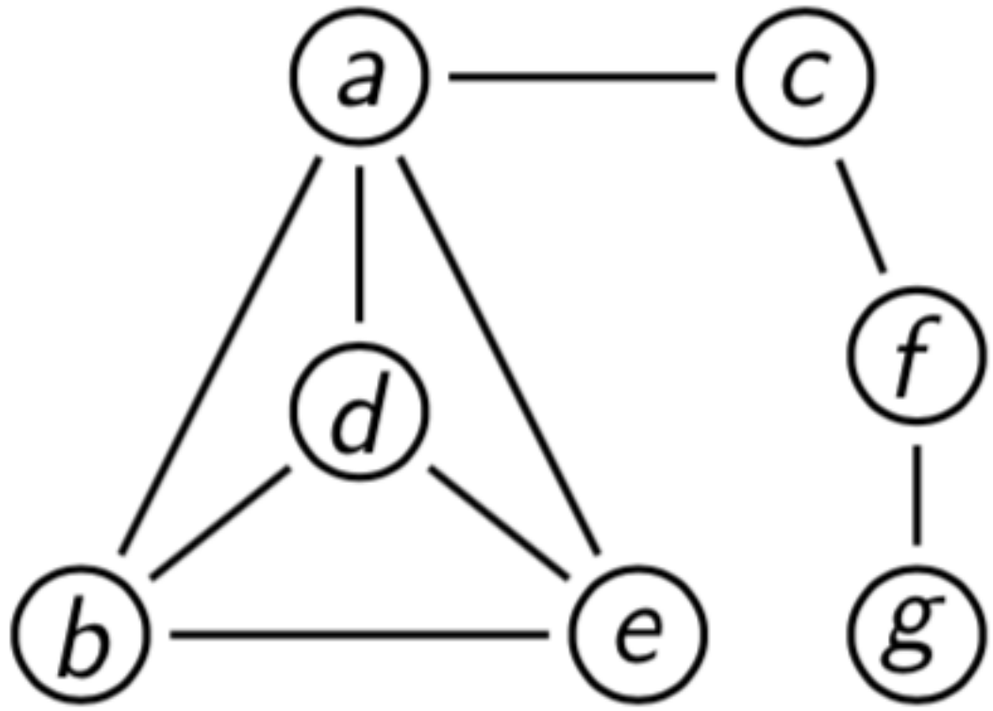
For directed graphs:

In-degree of v: number of edges going to v

Out-degree of v: number of edges leaving from v

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

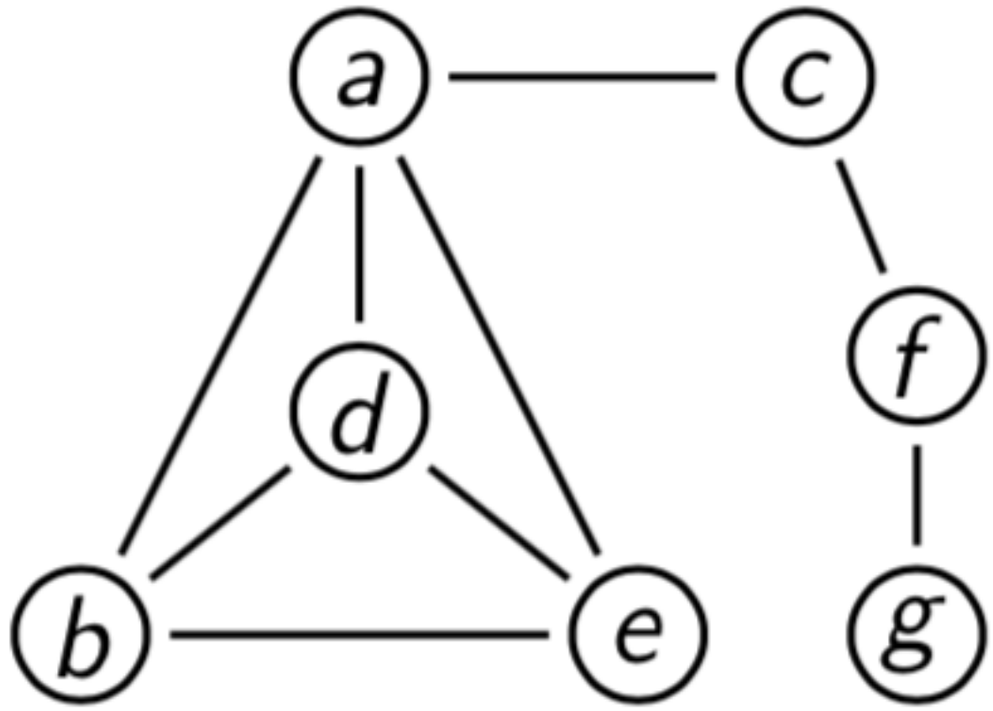
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

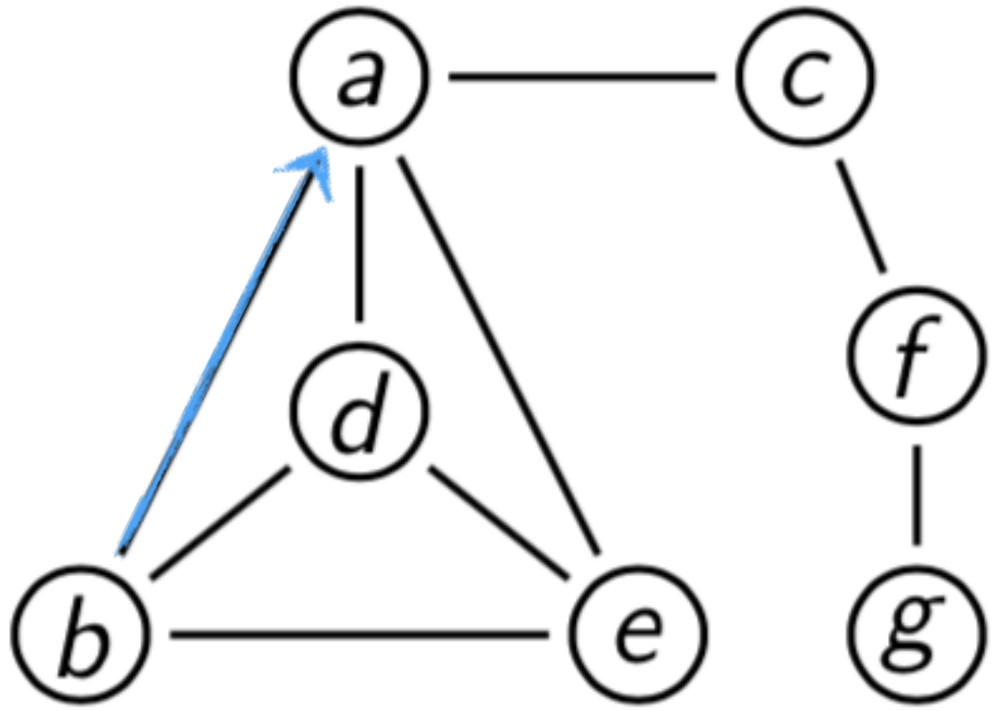
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

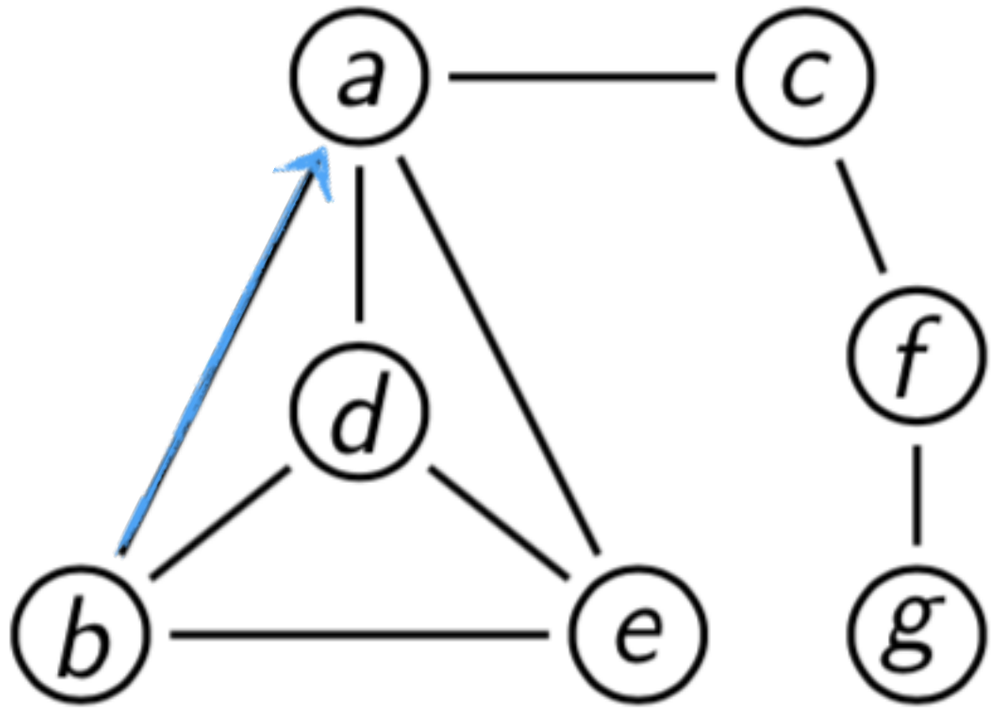
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b, a



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

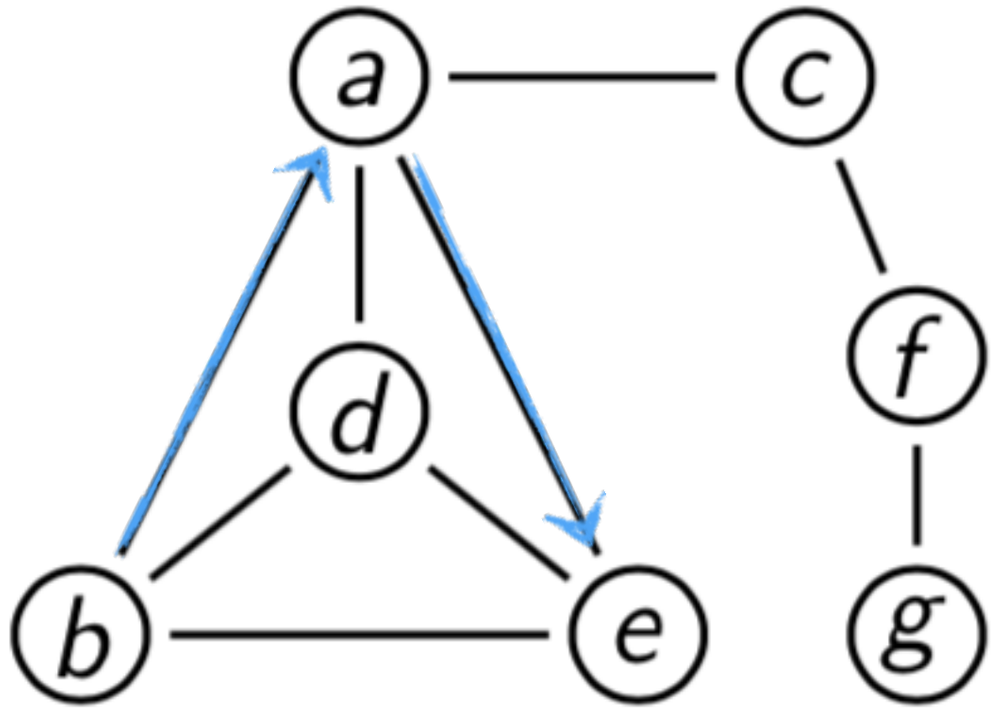
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b, a



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

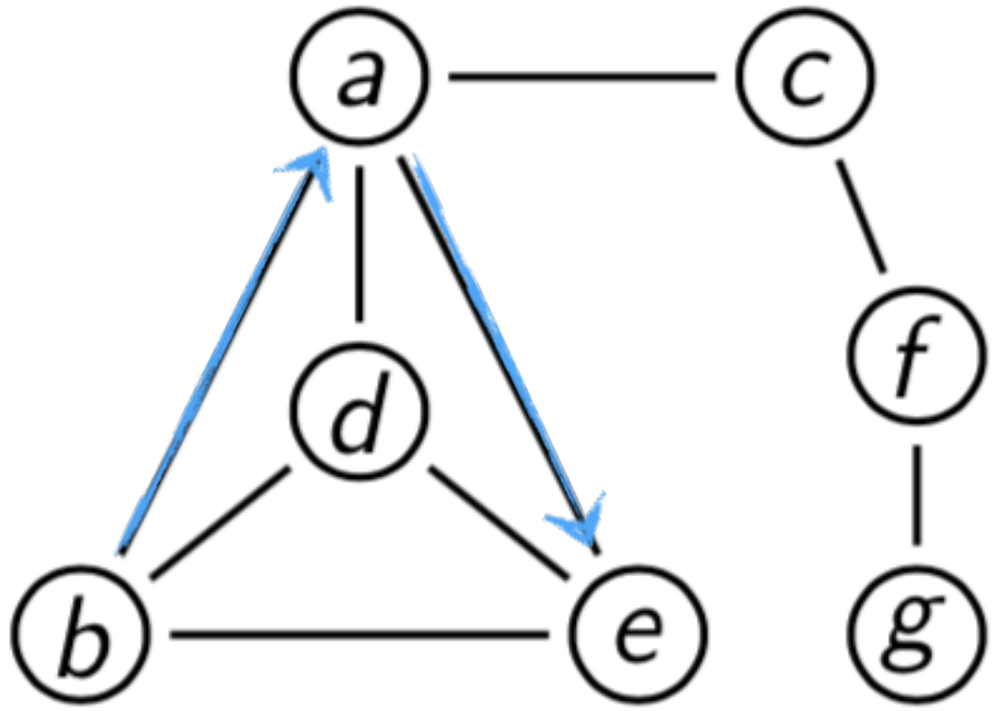
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b, a, e



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

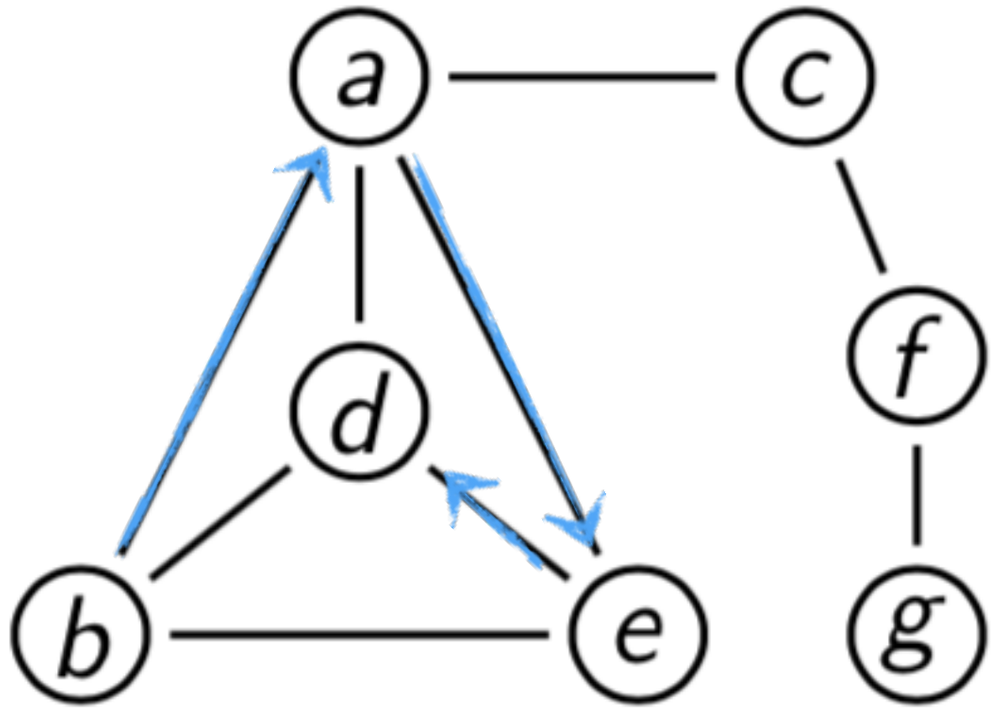
The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:
b, a, e



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

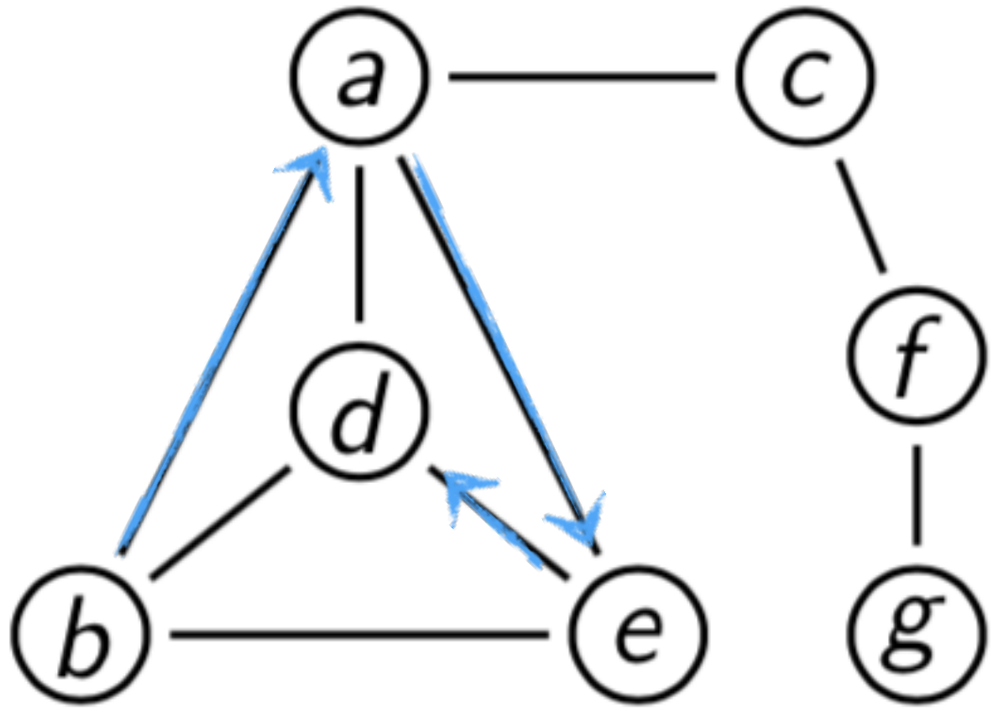
A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:

b, a, e, d



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

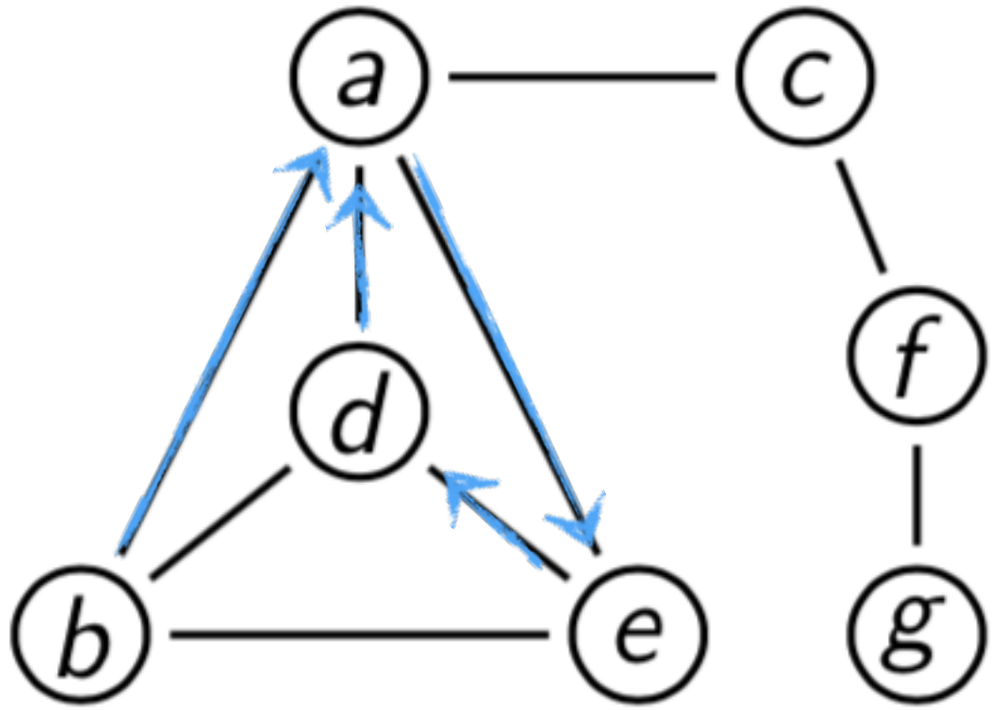
A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:

b, a, e, d



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

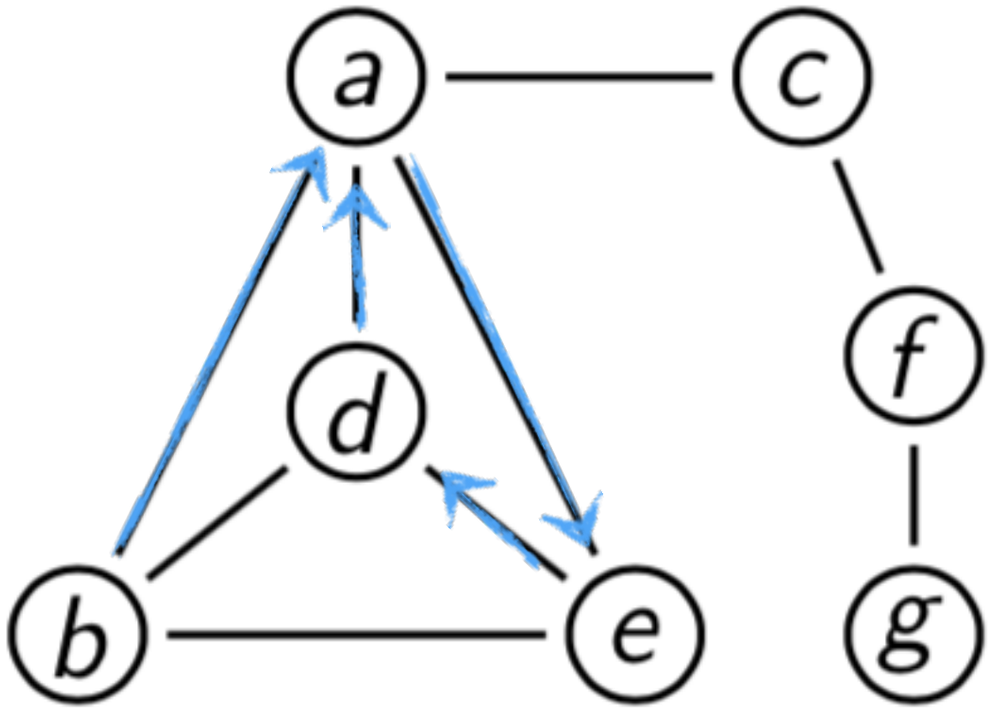
A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:

b, a, e, d, a



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

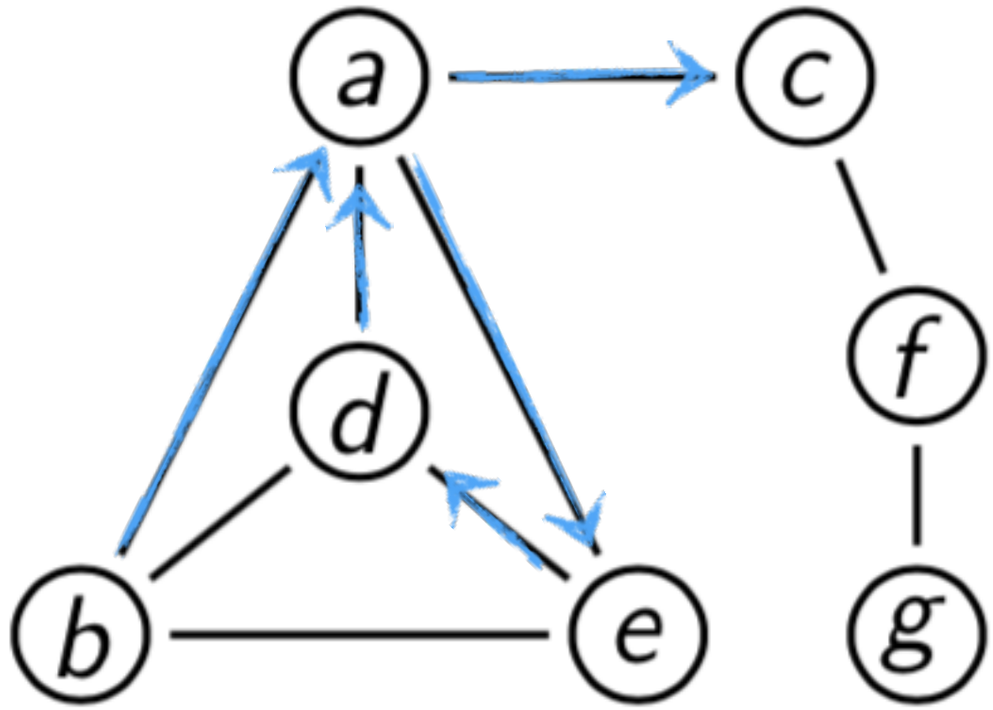
A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:

b, a, e, d, a



A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

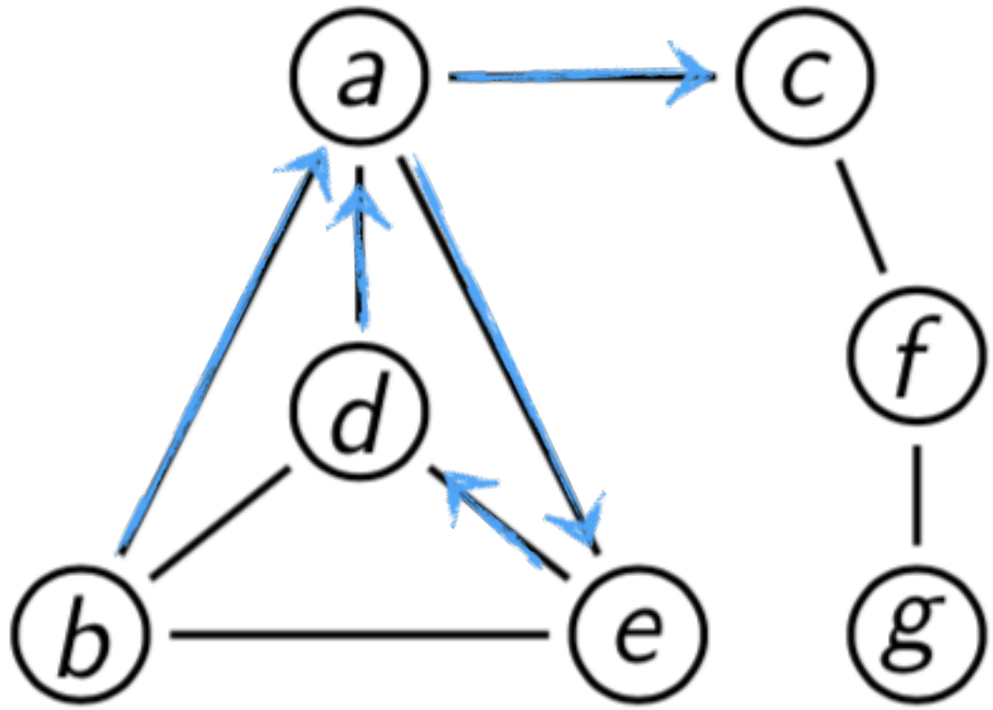
A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Paths

A path:

b, a, e, d, a, c



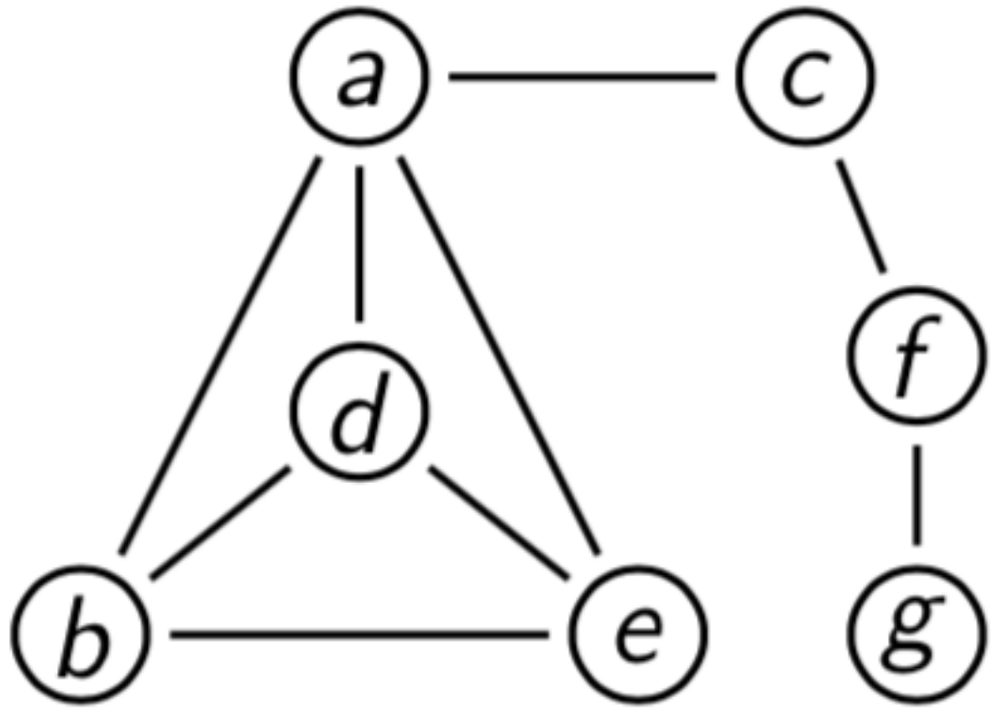
A **path** in ⟨V, E⟩ is a sequence of nodes $v_0,v_1,\ldots,v_k$ from V, so that each $(v_i,v_{i+1}) \in E$.

The path $v_0,v_1,\ldots,v_k$ has **length** k.

A **simple** path is one that has no repeated nodes.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles



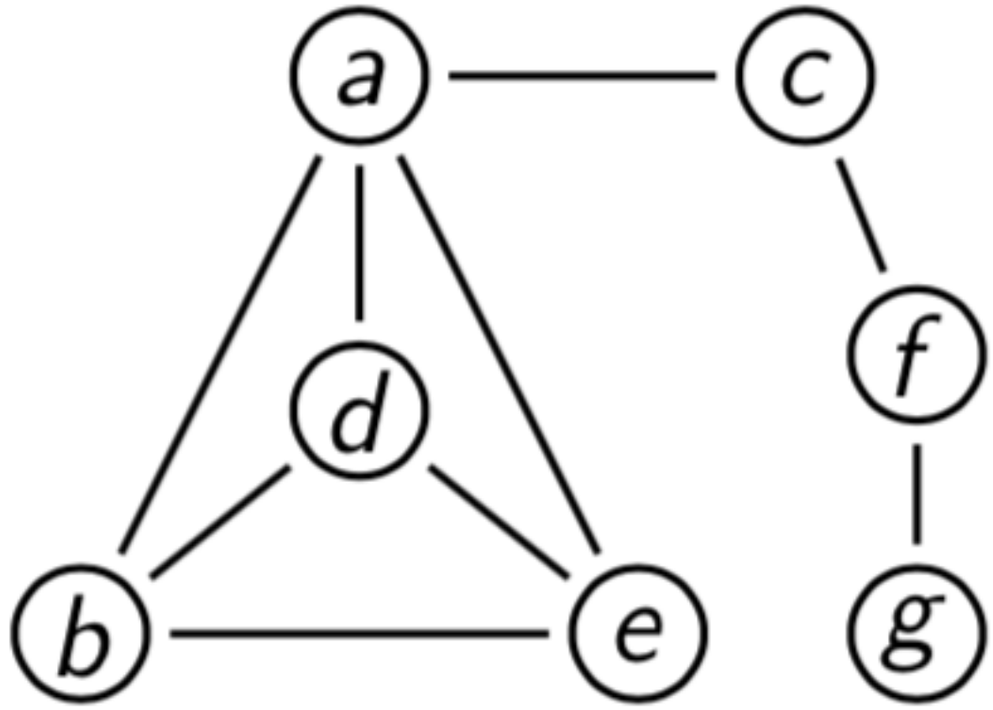
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b



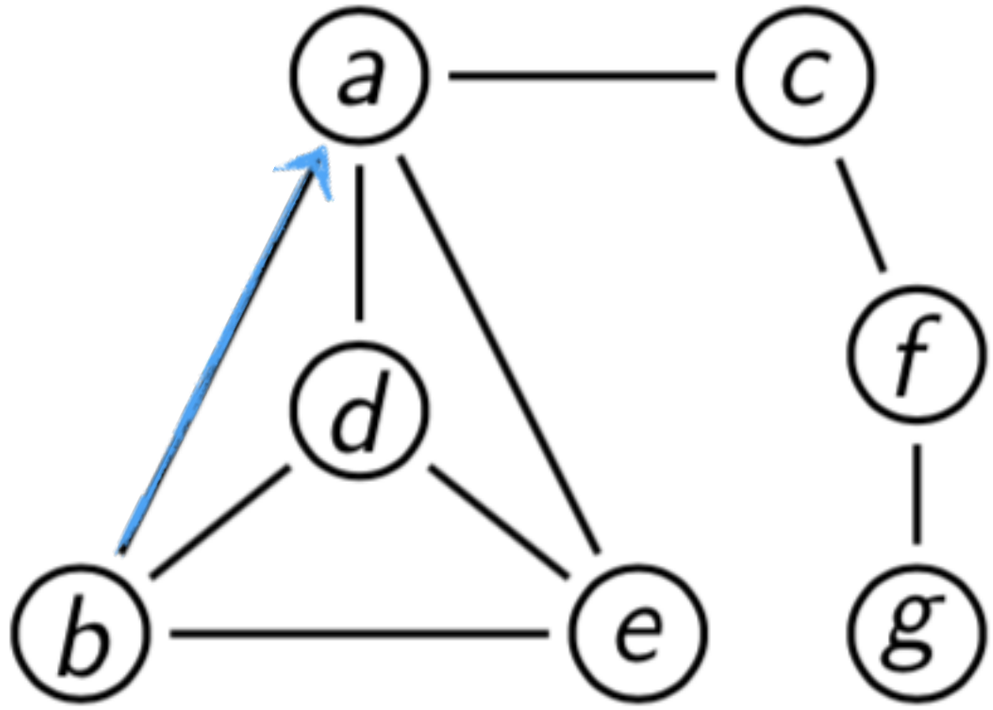
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b



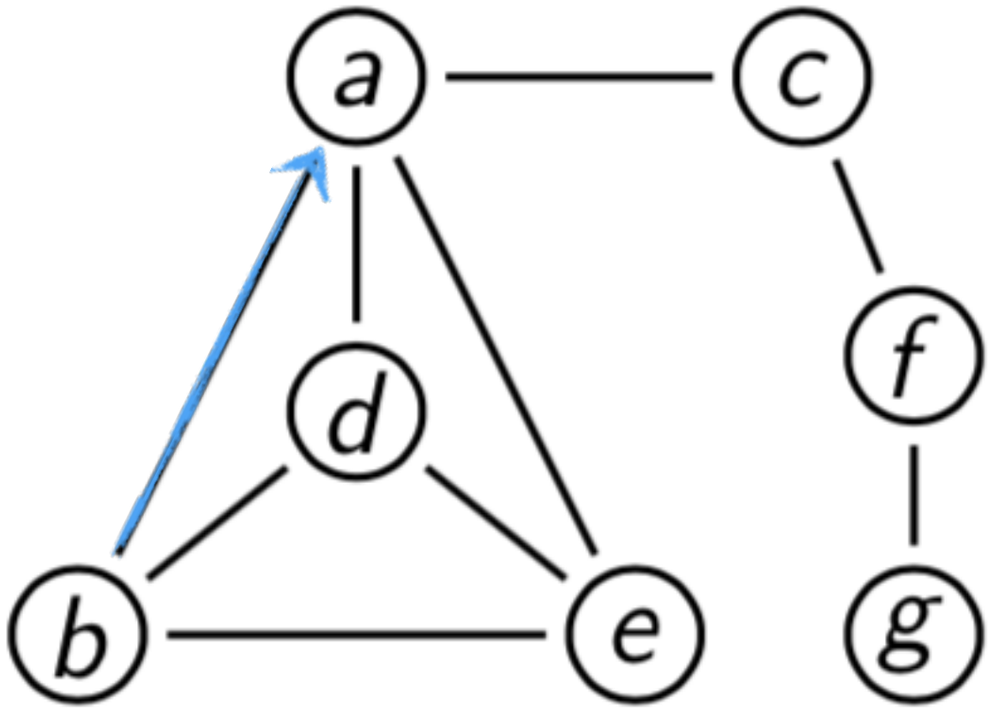
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b, a



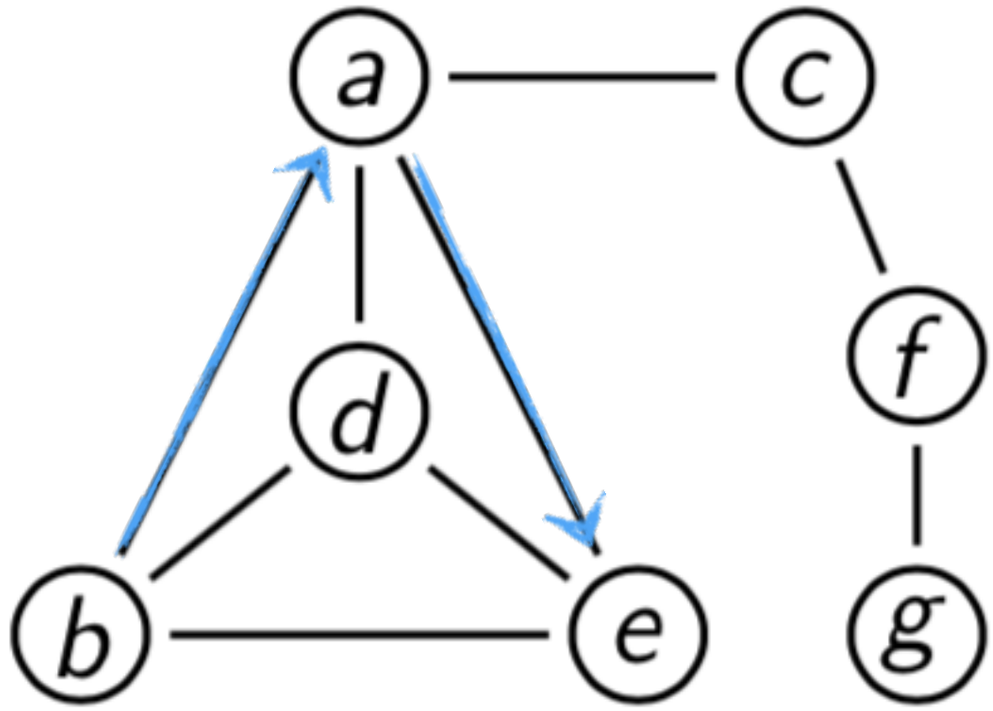
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b, a



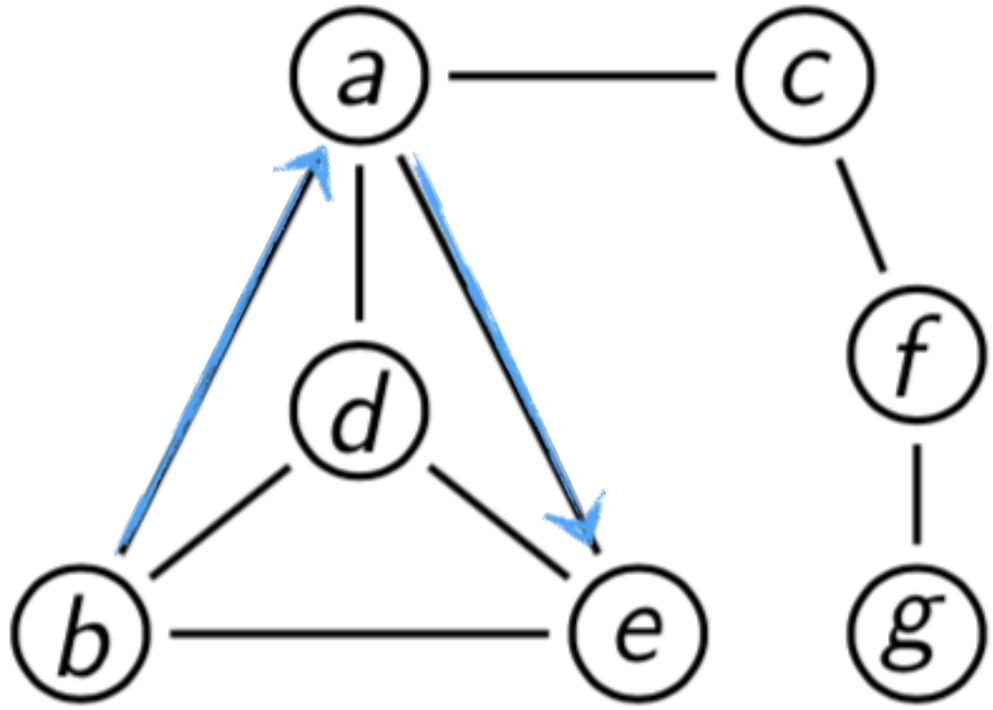
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b, a, e



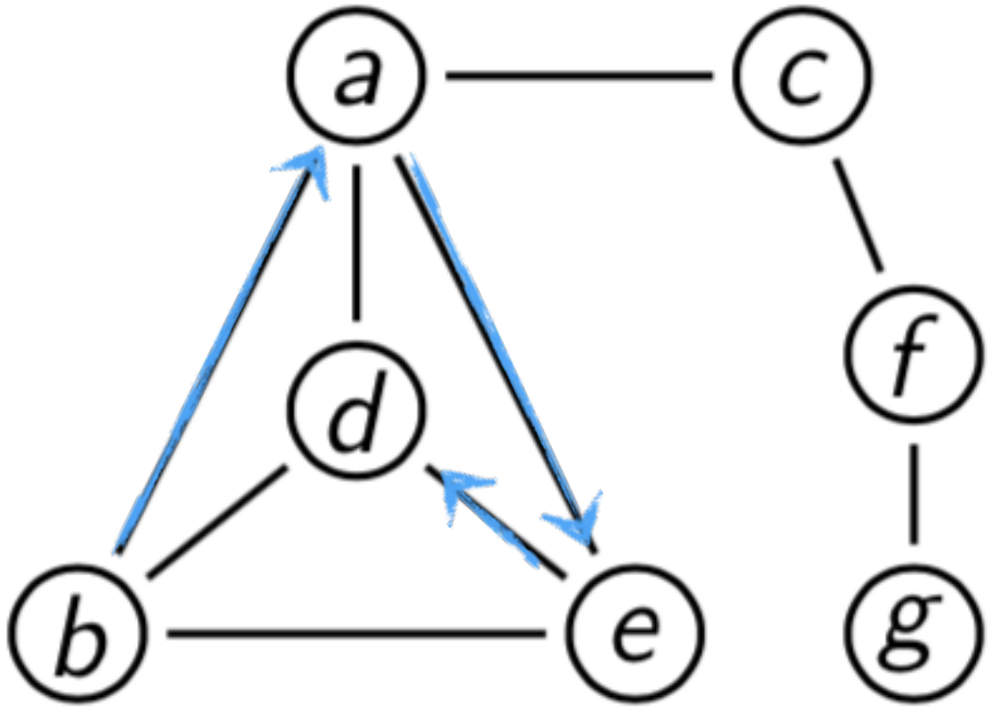
A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:
b, a, e



A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

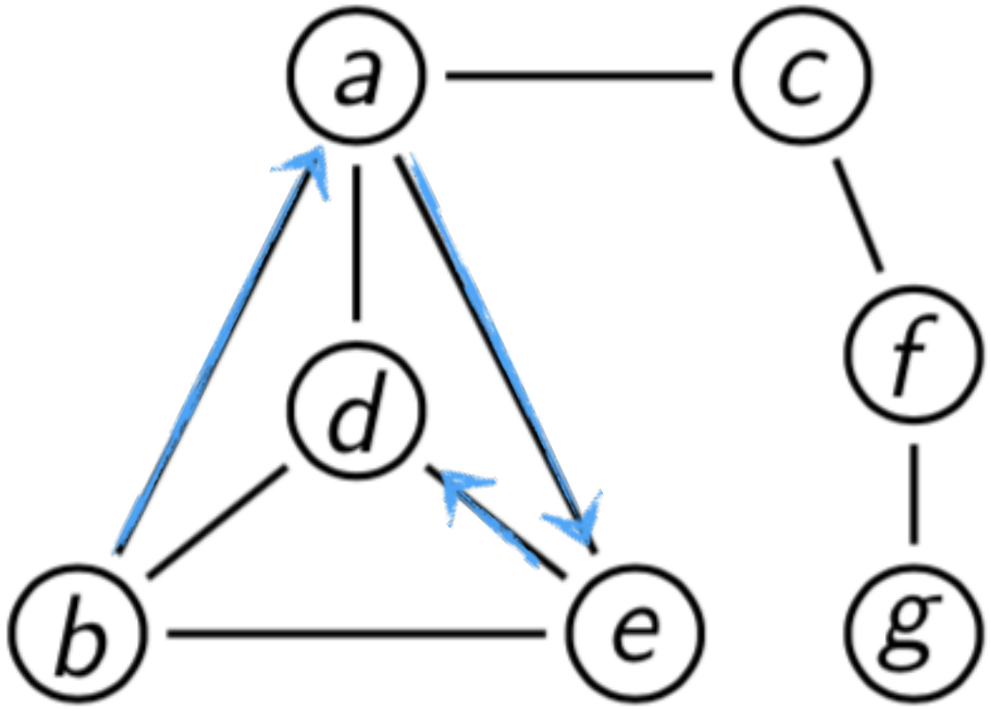
(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:

b, a, e, d



A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

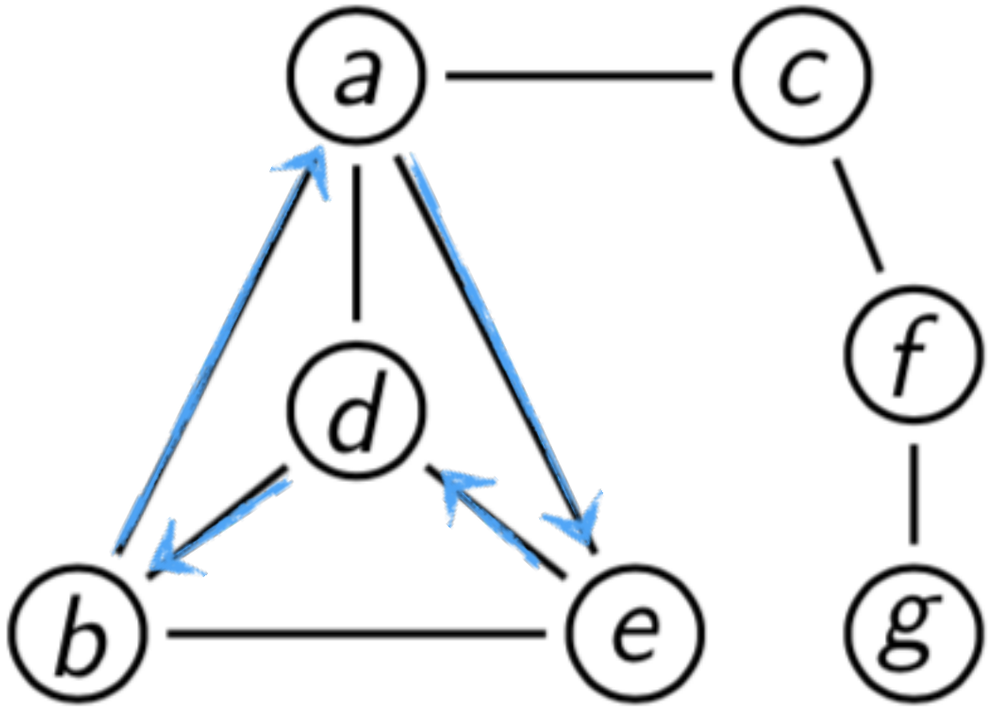
(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:

b, a, e, d



A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

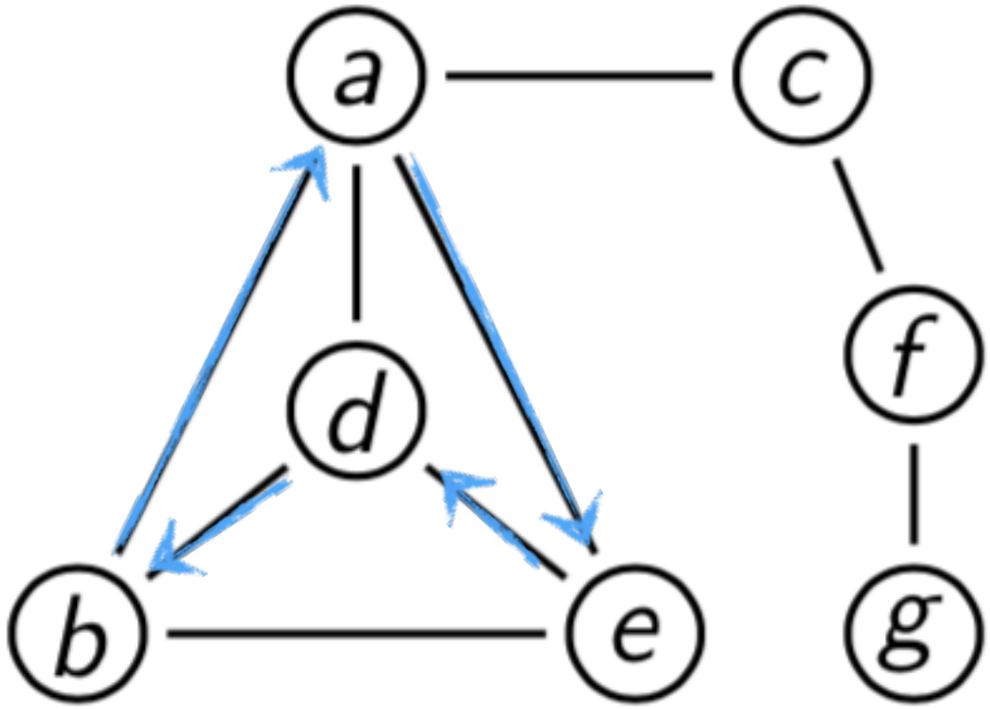
(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cycles

A cycle:

b, a, e, d, b

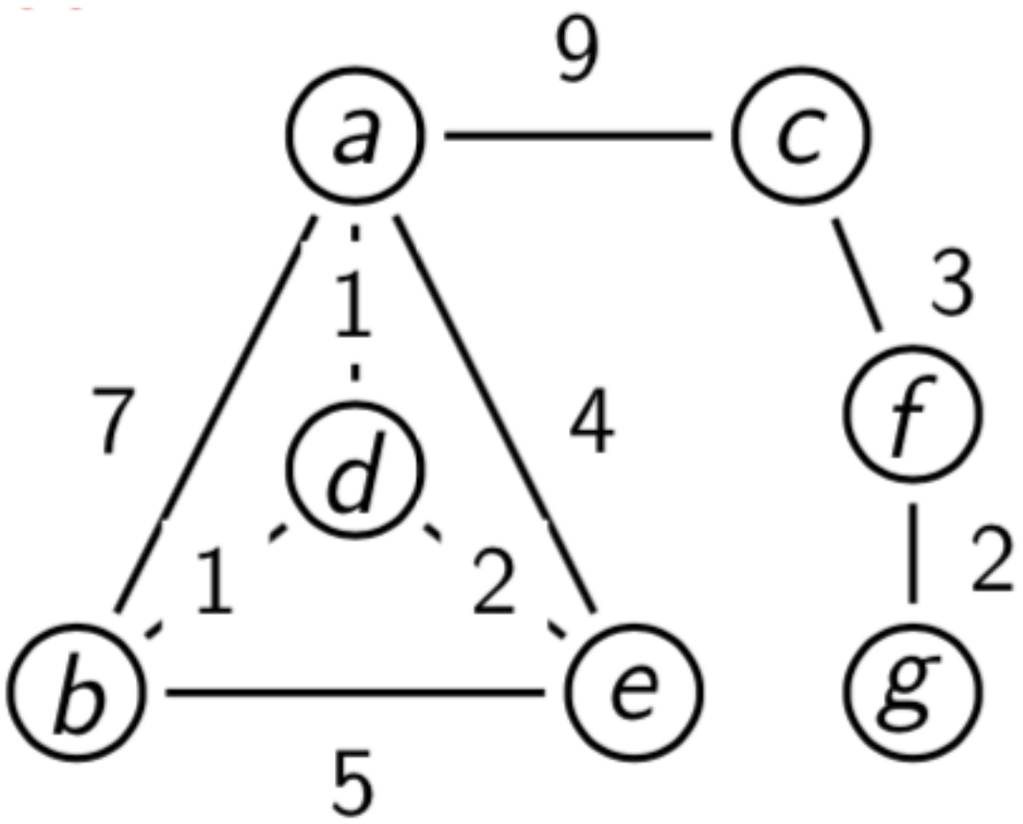


A **cycle** is a path that starts and finishes at the **same** node and does not traverse the same edge more than once.

(Cycles turn out to be very important for lots of applications.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Weighted Graphs

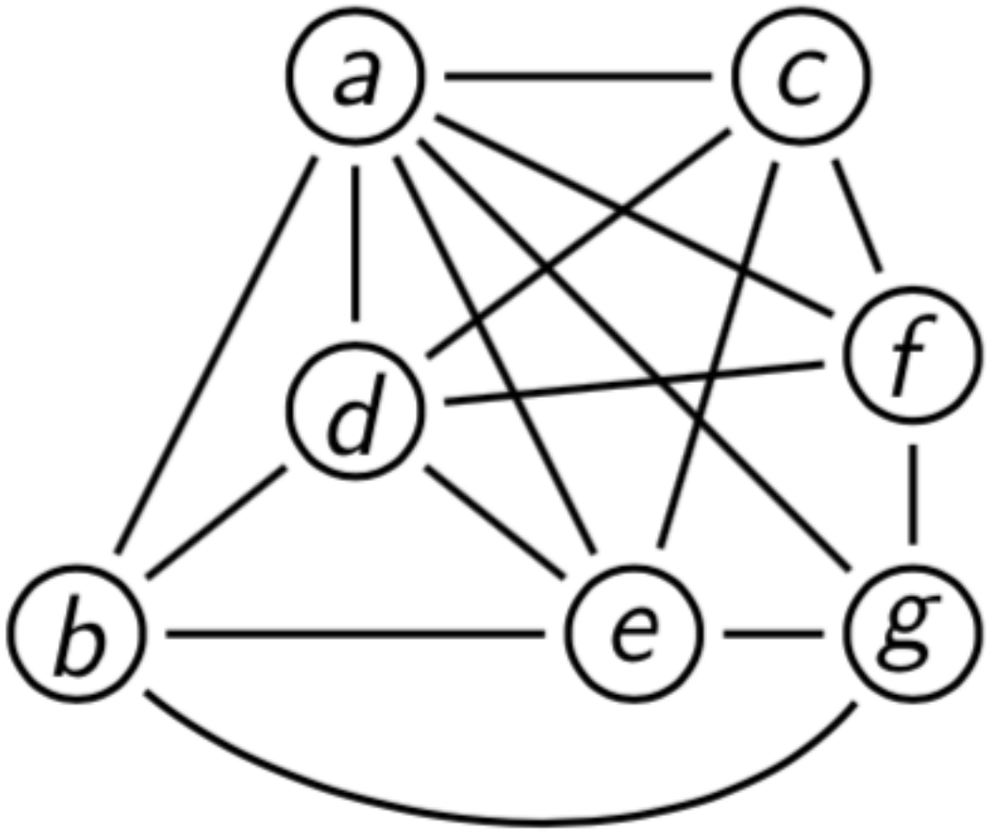


Each edge (v,u) has a **weight** indicating some information about that connection from v to u

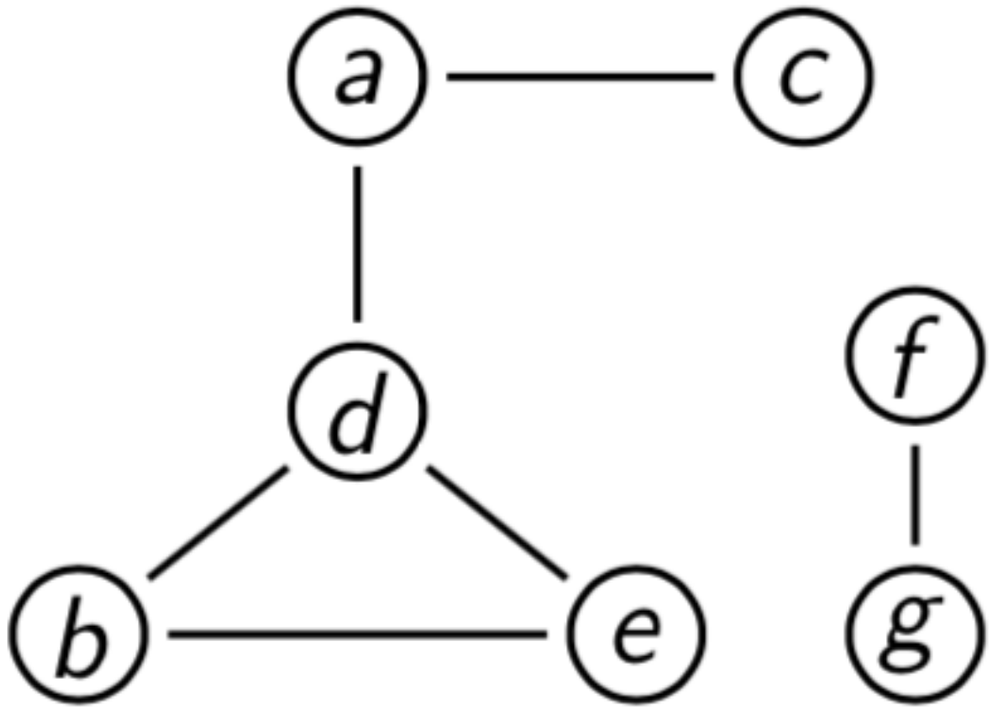
The information depends on what the graph represents: network congestion, physical distance, cost, etc.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Dense vs Sparse Graphs



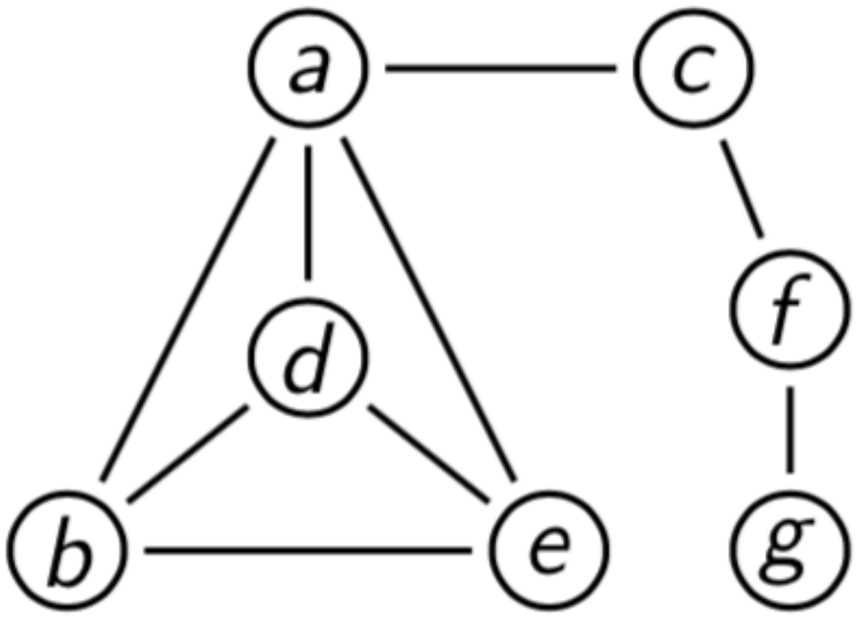
Dense Graph
(lots of edges, relative to number of nodes)



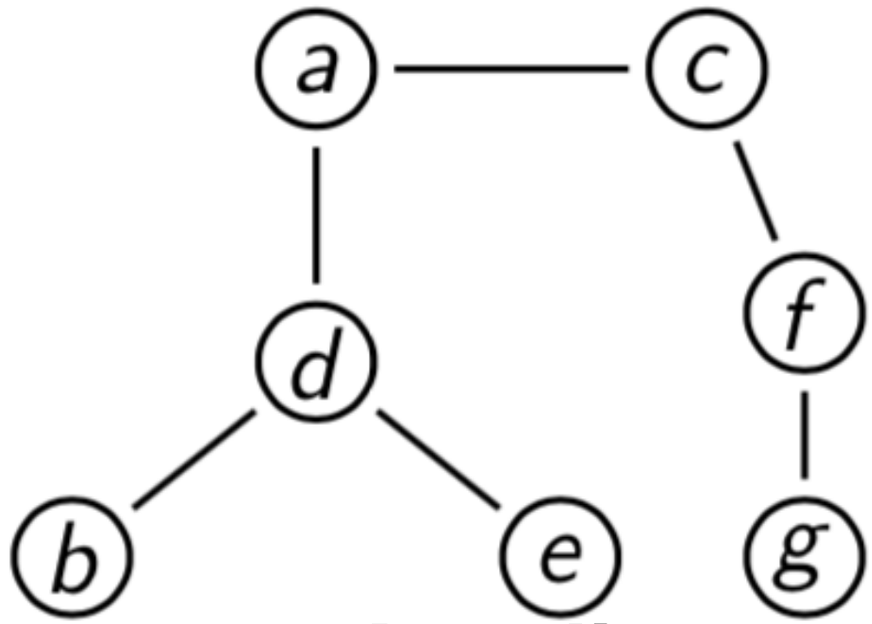
Sparse Graph
(few edges, relative to number of nodes)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Cyclic vs Acyclic



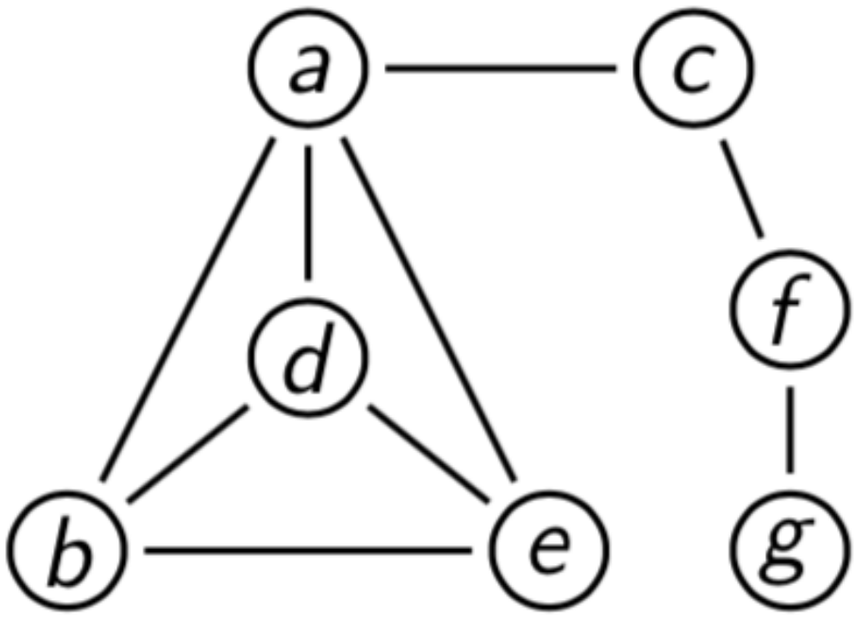
Cyclic



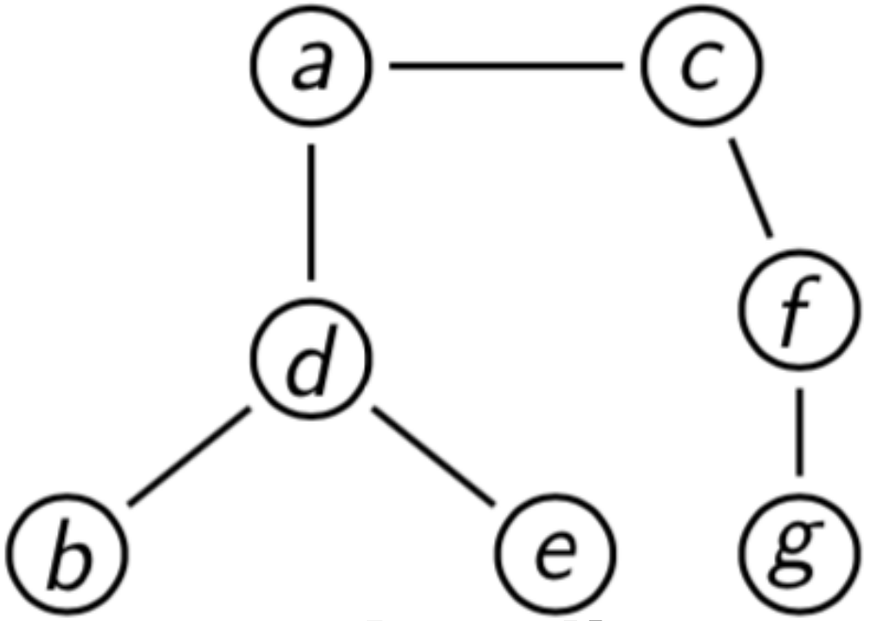
Acyclic

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

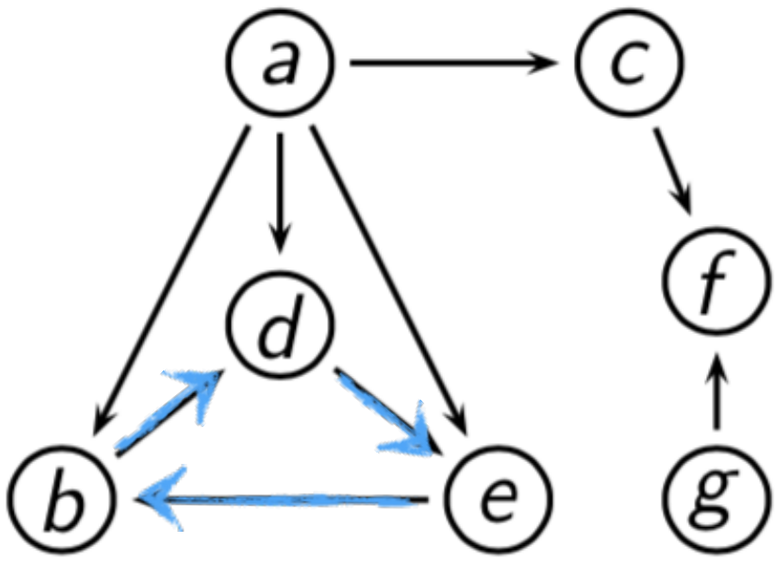
Cyclic vs Acyclic



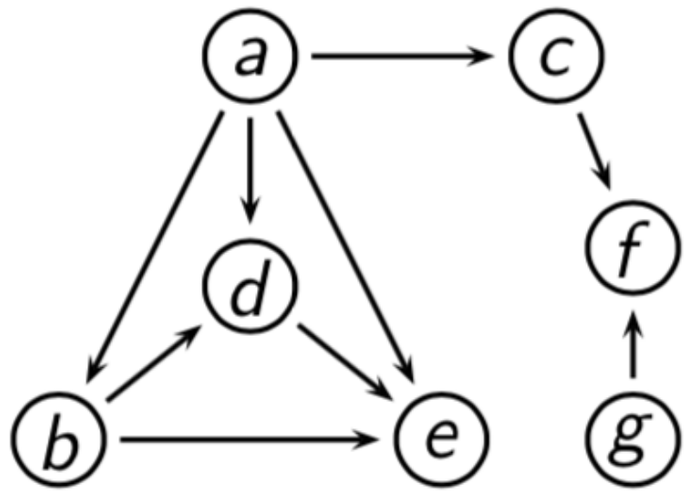
Cyclic



Acyclic



Directed Cyclic

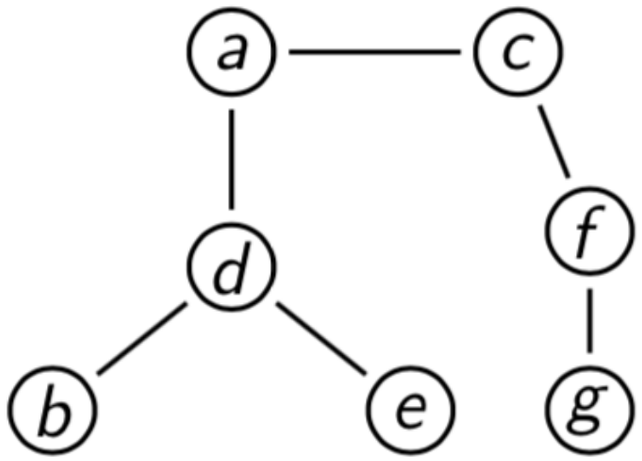


Directed Acyclic Graph (a dag)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Rooted Trees

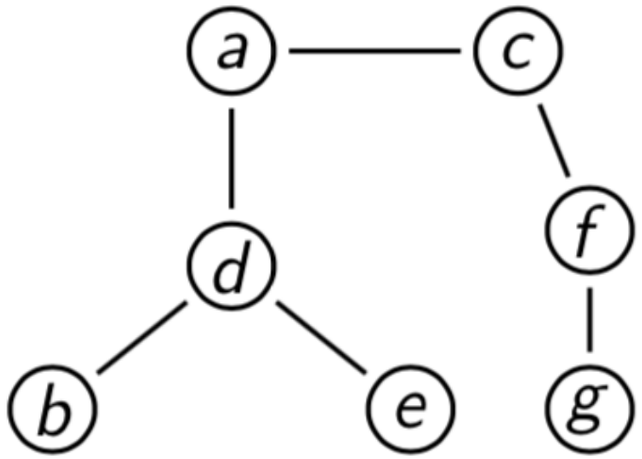
A (free) **tree** is a connected, acyclic graph, e.g.



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Rooted Trees

A (free) **tree** is a connected, acyclic graph, e.g.

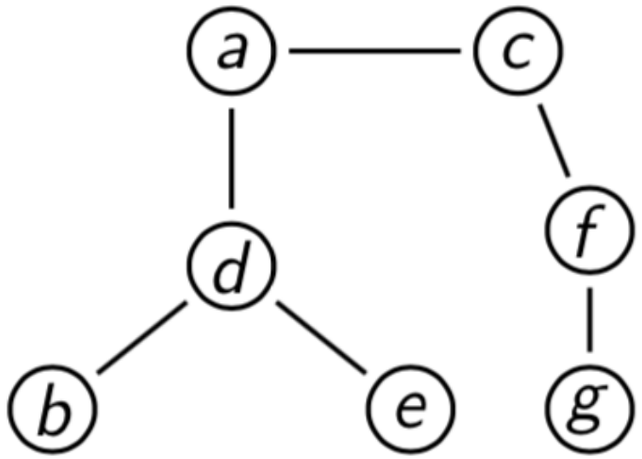


A **rooted tree** is a tree with one node (the **root**) identified as special. Every other node is reachable from the root node.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Rooted Trees

A (free) **tree** is a connected, acyclic graph, e.g.



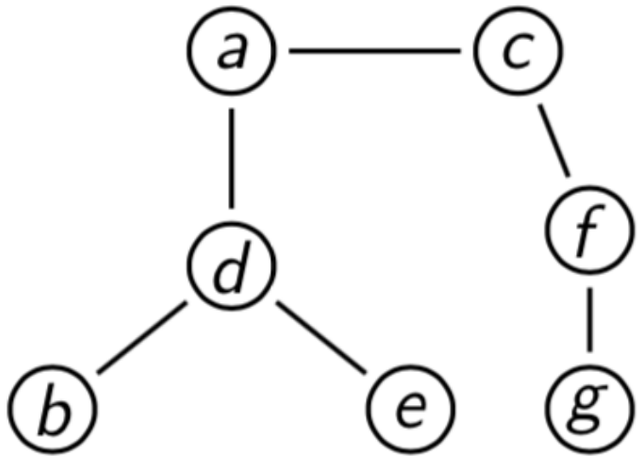
A **rooted tree** is a tree with one node (the **root**) identified as special. Every other node is reachable from the root node.

When the root is removed, a set of rooted (sub-)trees remain.

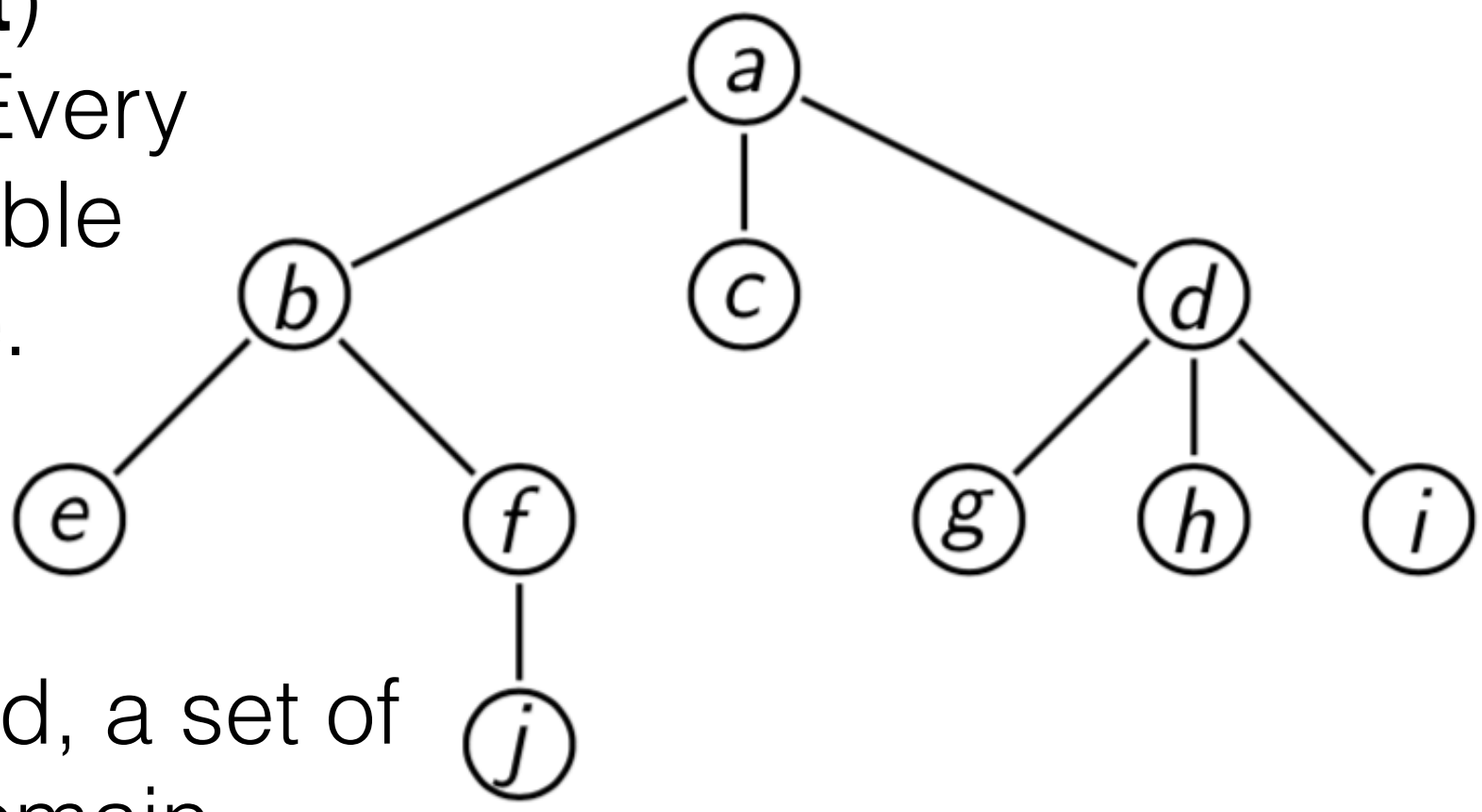
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Rooted Trees

A (free) **tree** is a connected, acyclic graph, e.g.



A **rooted tree** is a tree with one node (the **root**) identified as special. Every other node is reachable from the root node.

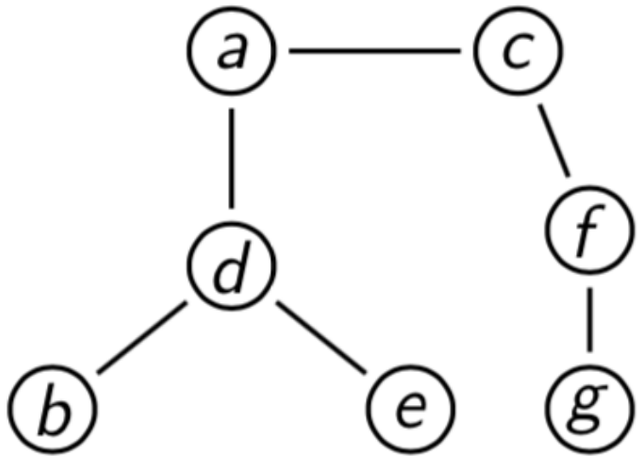


When the root is removed, a set of rooted (sub-)trees remain.

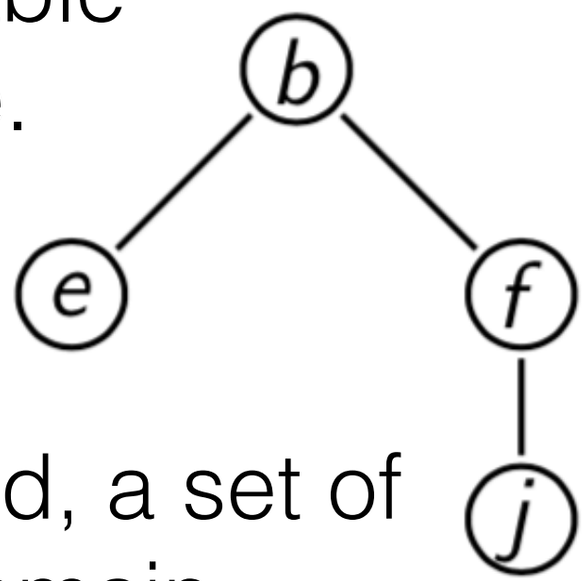
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Rooted Trees

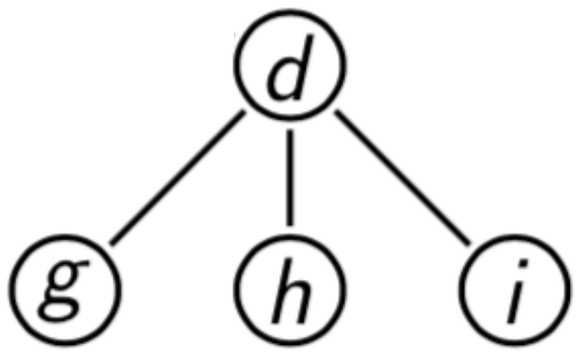
A (free) **tree** is a connected, acyclic graph, e.g.



A **rooted tree** is a tree with one node (the **root**) identified as special. Every other node is reachable from the root node.







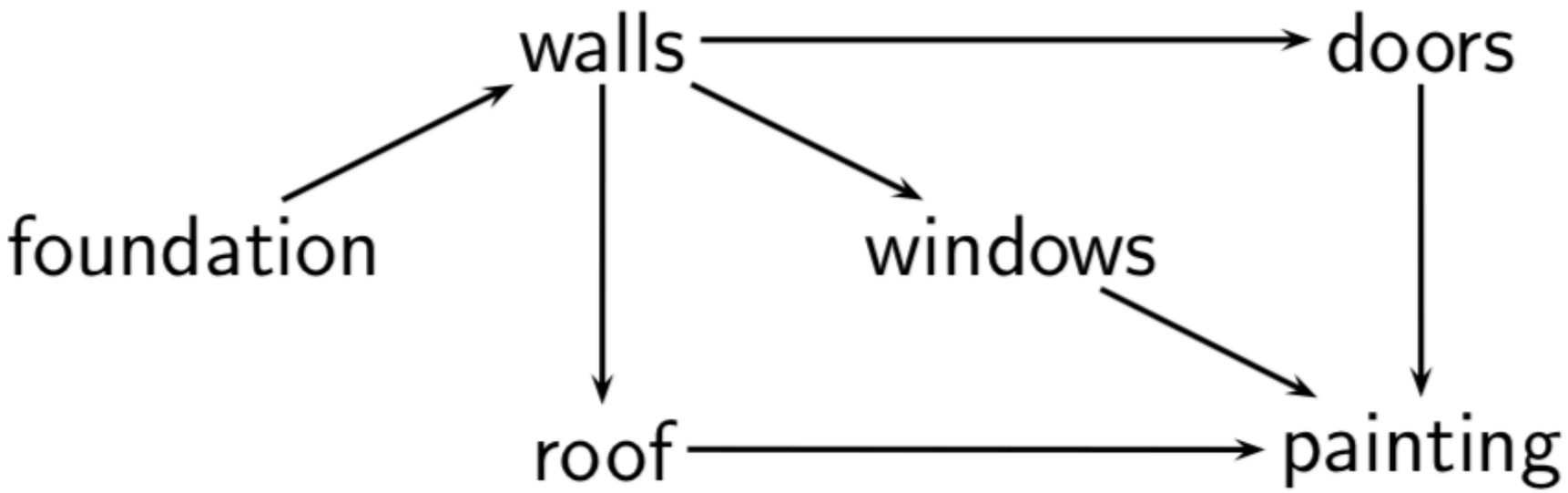
When the root is removed, a set of rooted (sub-)trees remain.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Graph algorithms are of great importance because so many different problem types can be abstracted to graph problems.

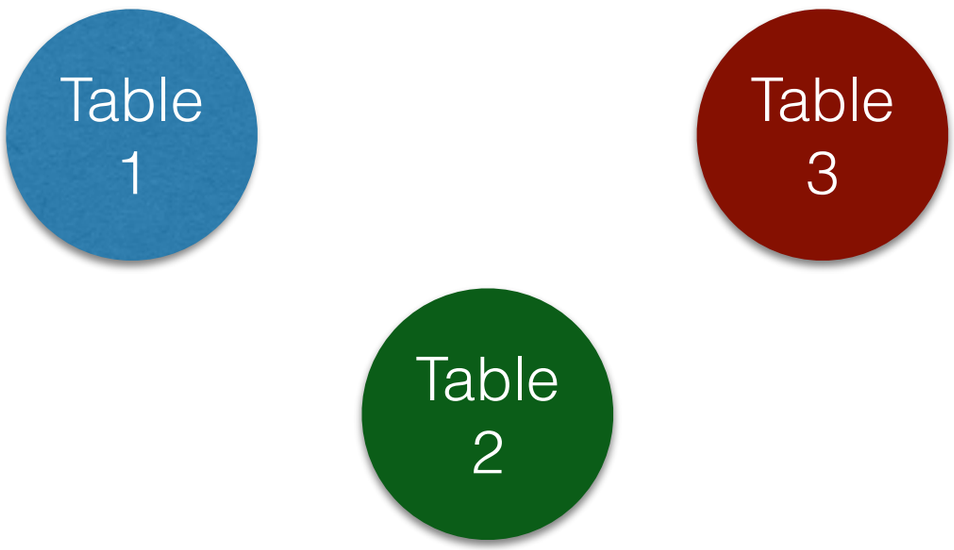
For example, directed graphs (they'd better be dags) are central in scheduling problems:



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Imagine I'm doing the seating plan for a wedding.

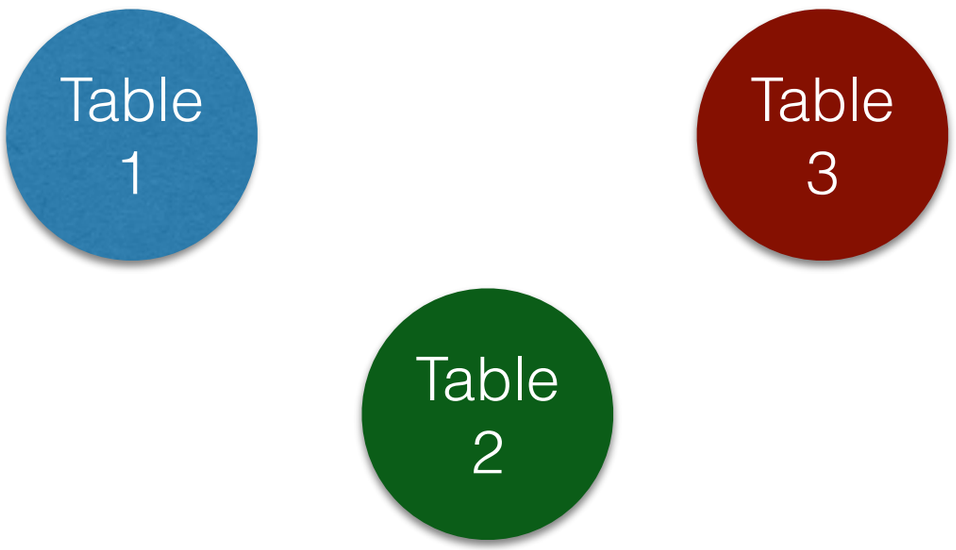


	Guest List
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Imagine I'm doing the seating plan for a wedding.



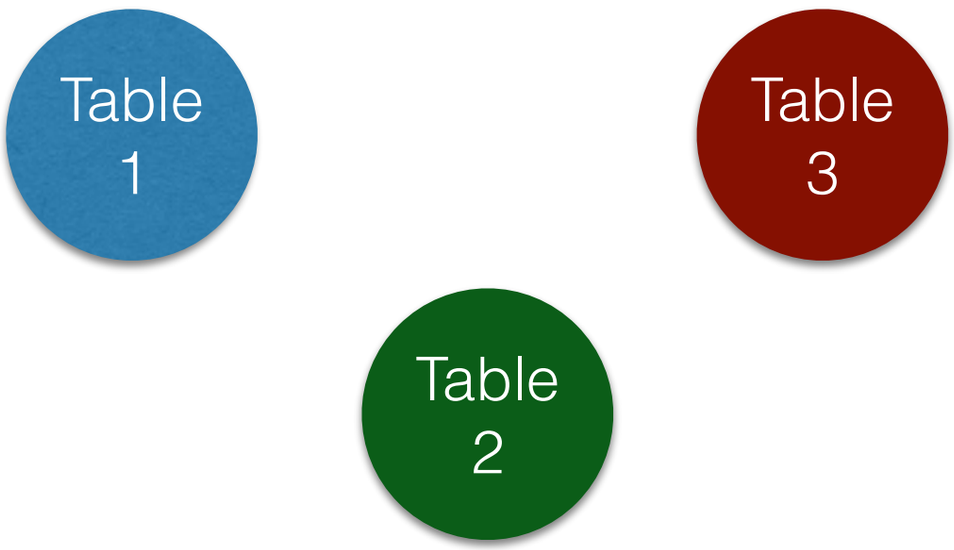


	Guest List
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Imagine I'm doing the seating plan for a wedding.

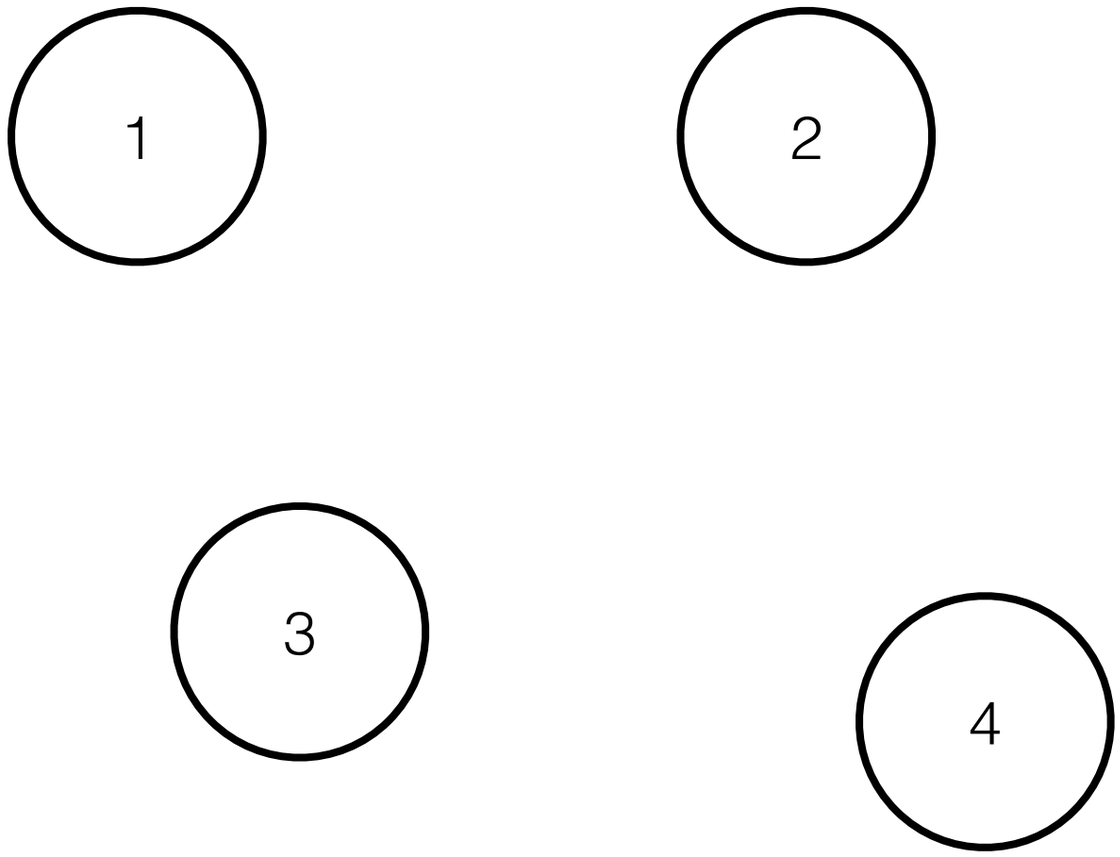


	Guest List
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.

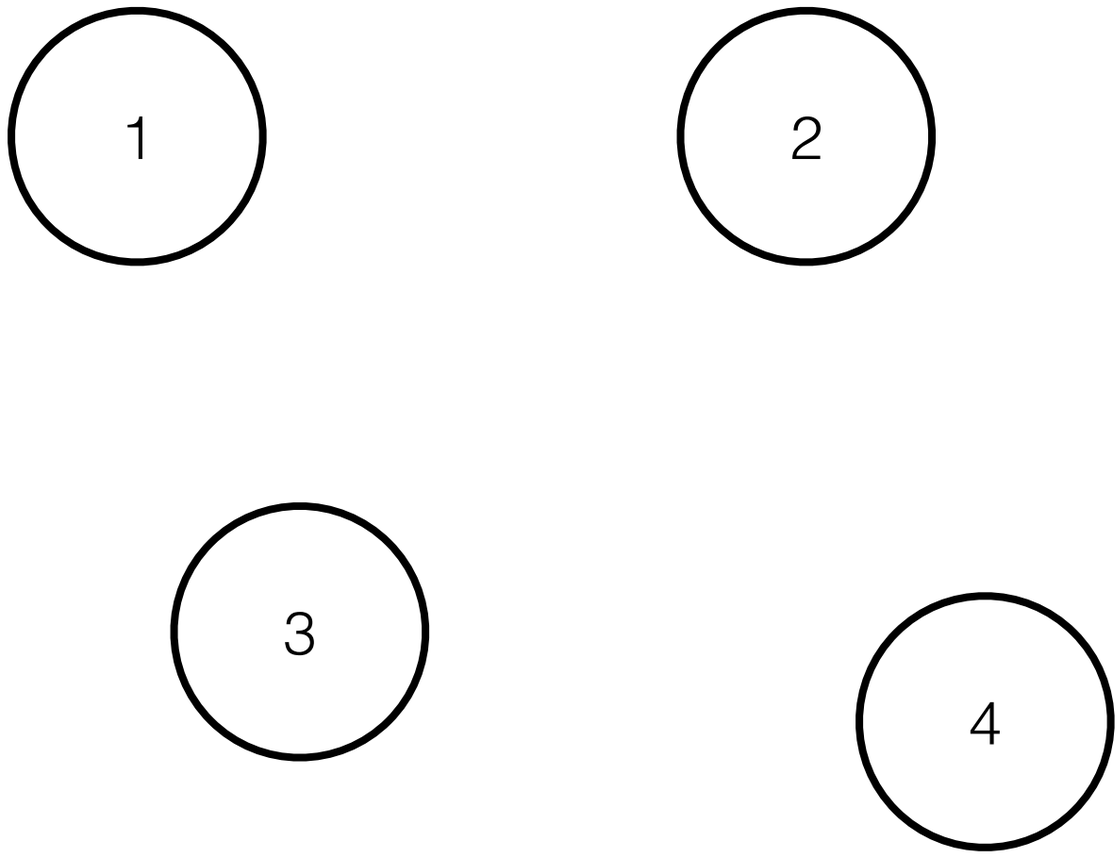


	Guest List
1	Aunt Marth
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.
An edge between v and u means v and u cannot sit together.

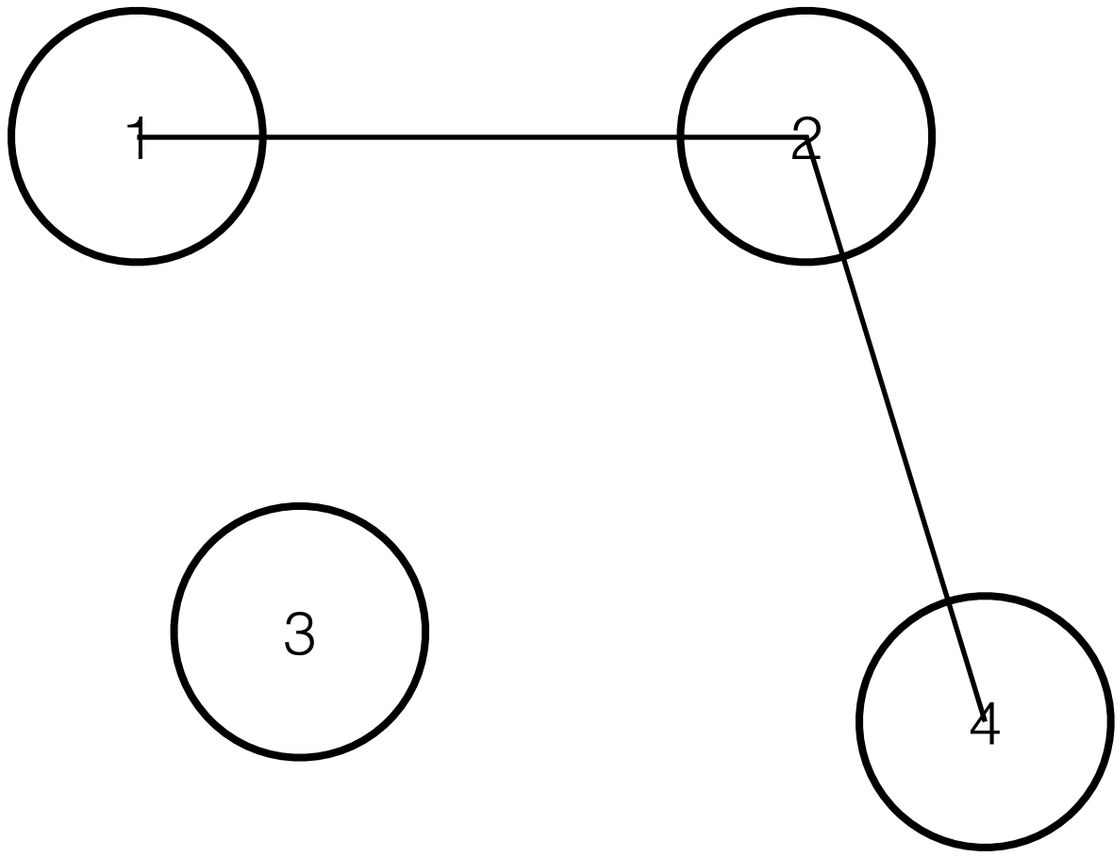


Guest List	
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.
An edge between v and u means v and u cannot sit together.

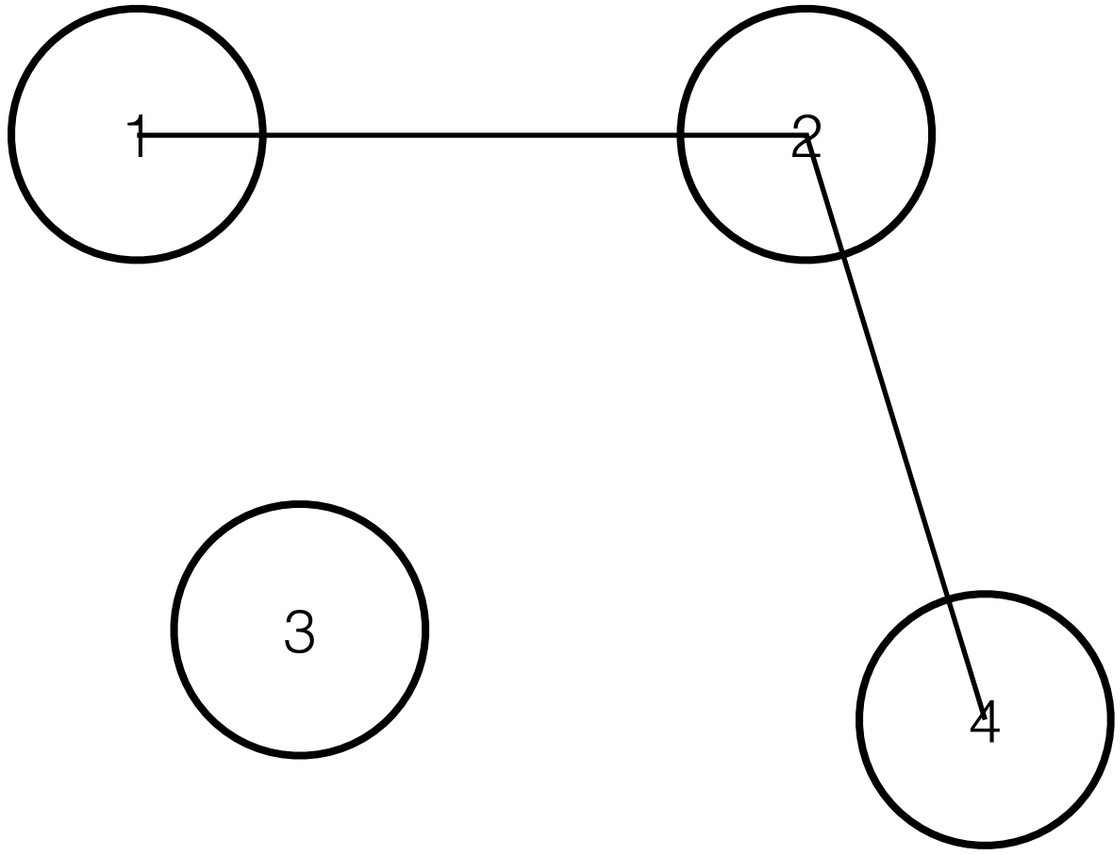


	Guest List
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.
An edge between v and u means v and u cannot sit together.



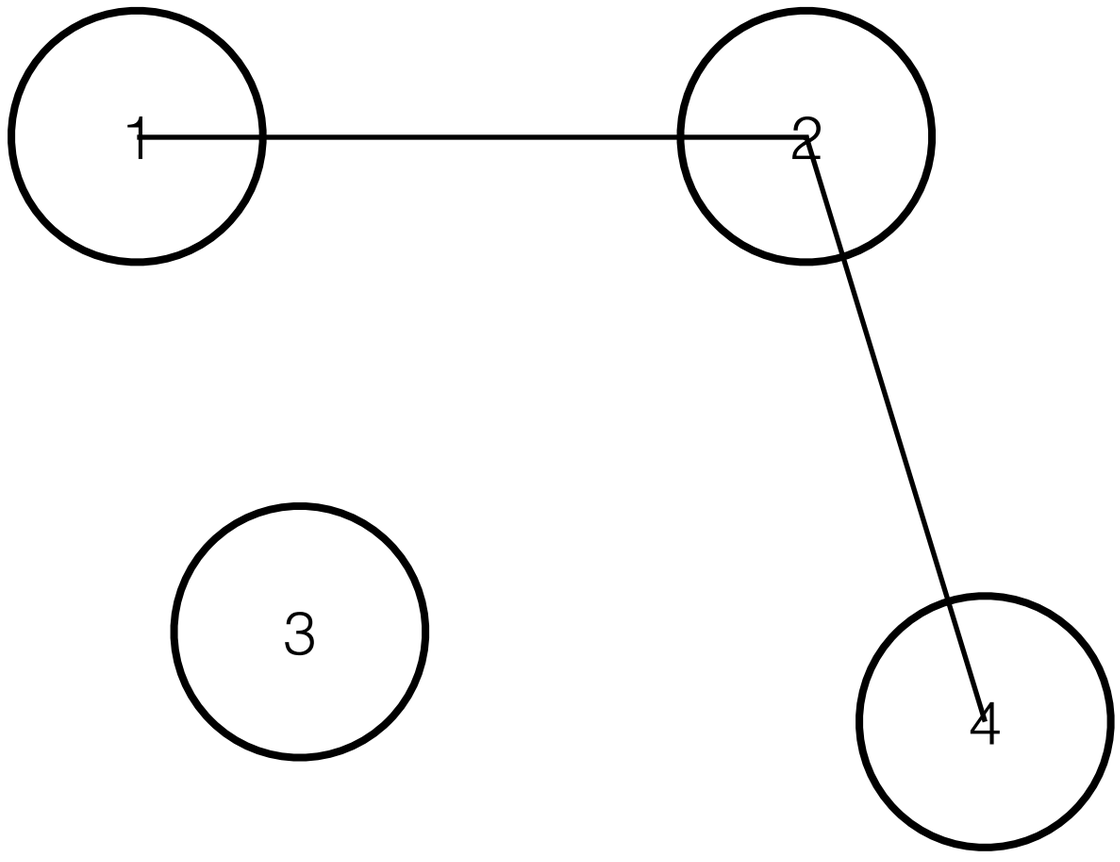


Guest List	
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.
An edge between v and u means v and u cannot sit together.



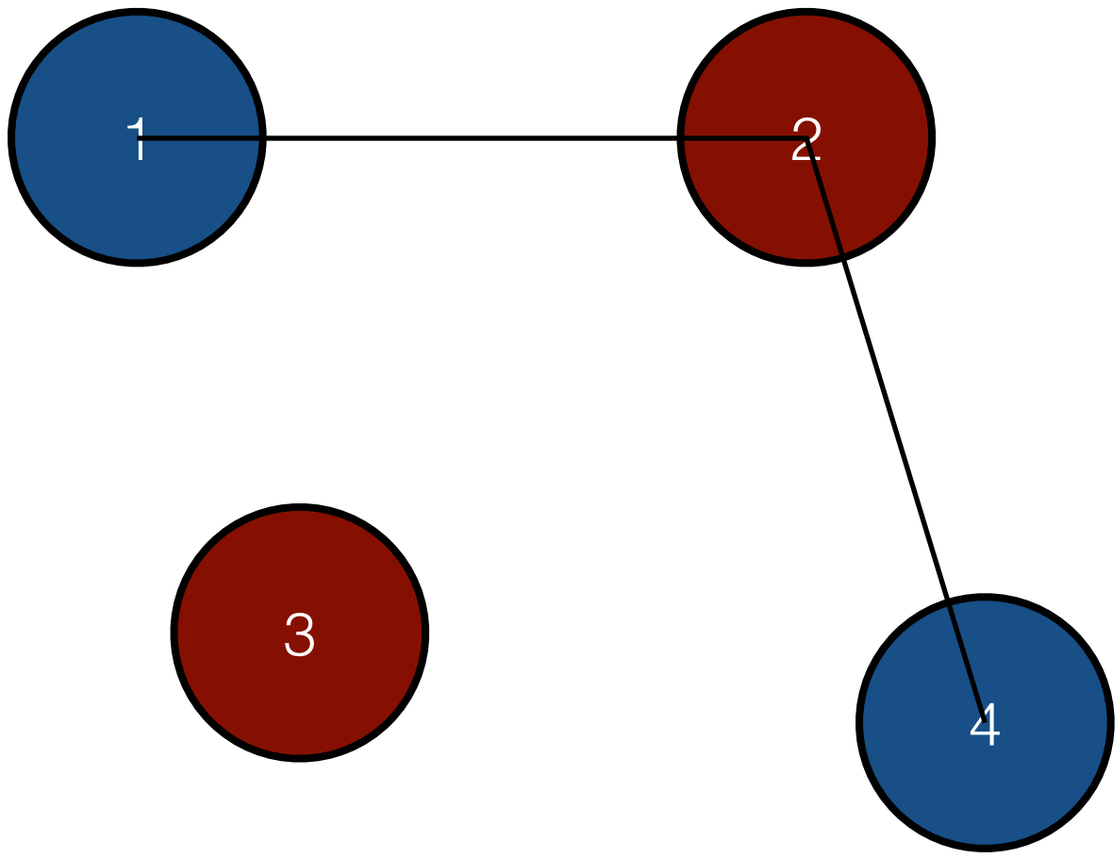
Now colour the nodes so that no two adjacent nodes have the same colour.

	Guest List
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

Each person becomes a node.
An edge between v and u means v and u cannot sit together.



Now colour the nodes so that no two adjacent nodes have the same colour.

Guest List	
1	Aunt Martha
2	Aunt Sally
3	Uncle Joe
4	Cousin Andy
…	….

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Modelling with Graphs

THE UNIVERSITY OF MELBOURNE

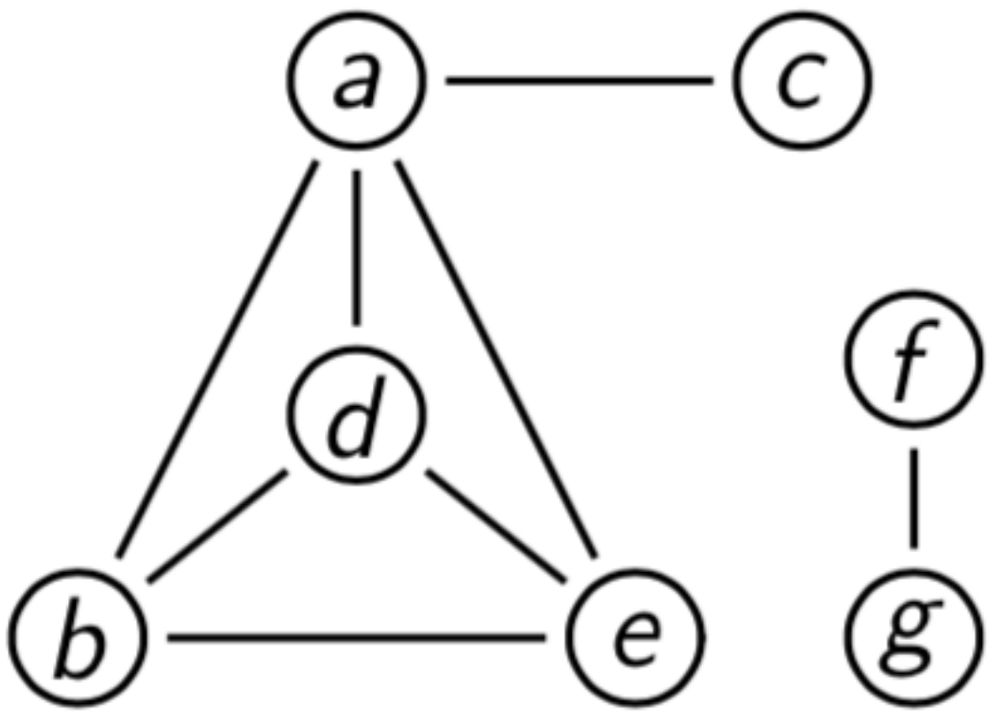
Seating planning with k tables can be **reduced** to the graph k-coloring problem:

Find, if possible, a colouring of nodes so that no two connected nodes get the same colour.

Lots of other problems can be reduced to graph colouring.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations: Undirected Graphs



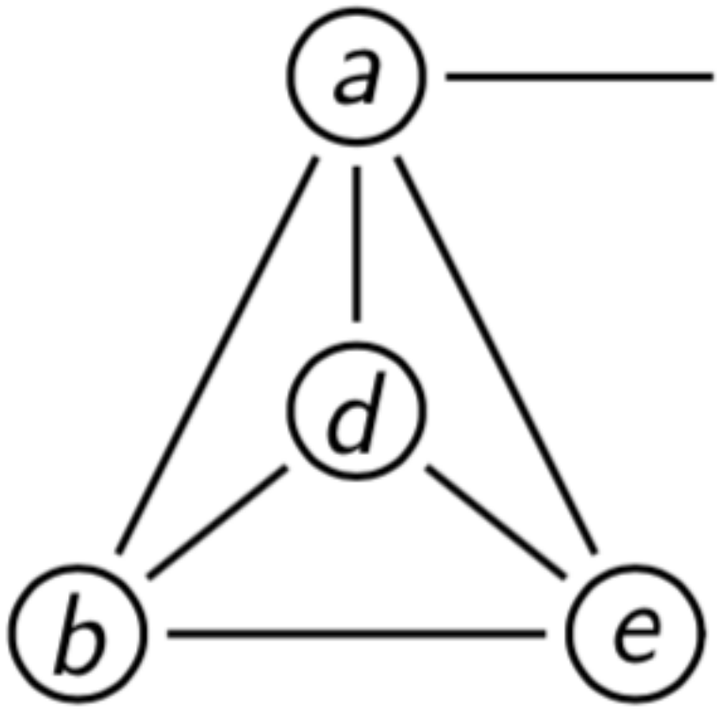
	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency Matrix

For an undirected graph, this matrix is **symmetric** about the diagonal.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations: Undirected Graphs



a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

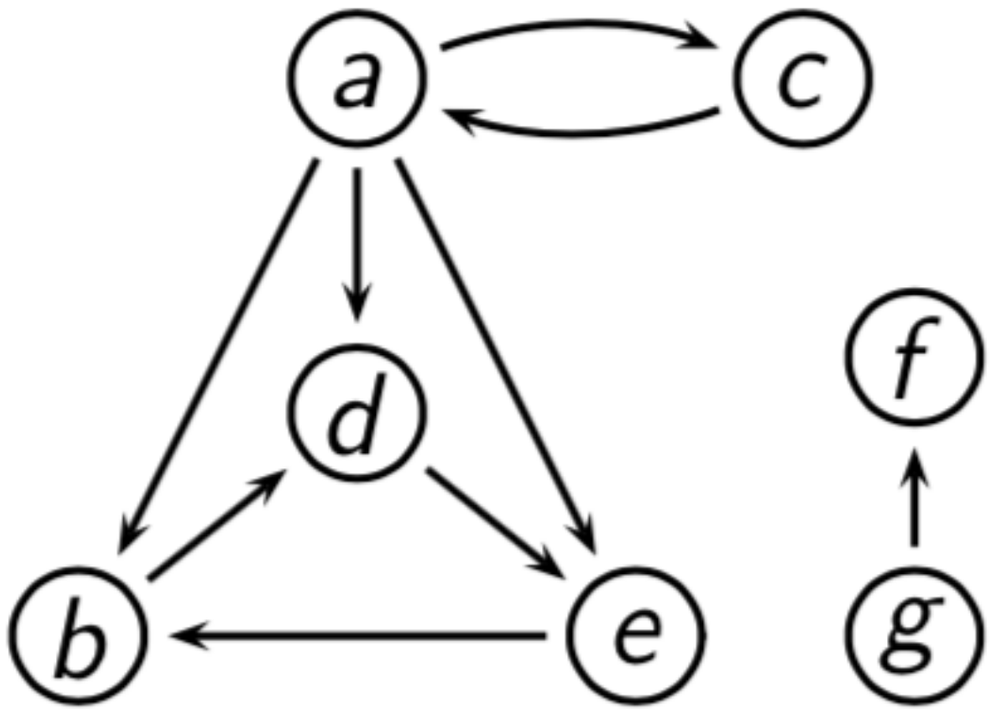
Adjacency List

An **array of linked lists**

(Assuming lists are kept in sorted order.)

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations: Directed Graphs

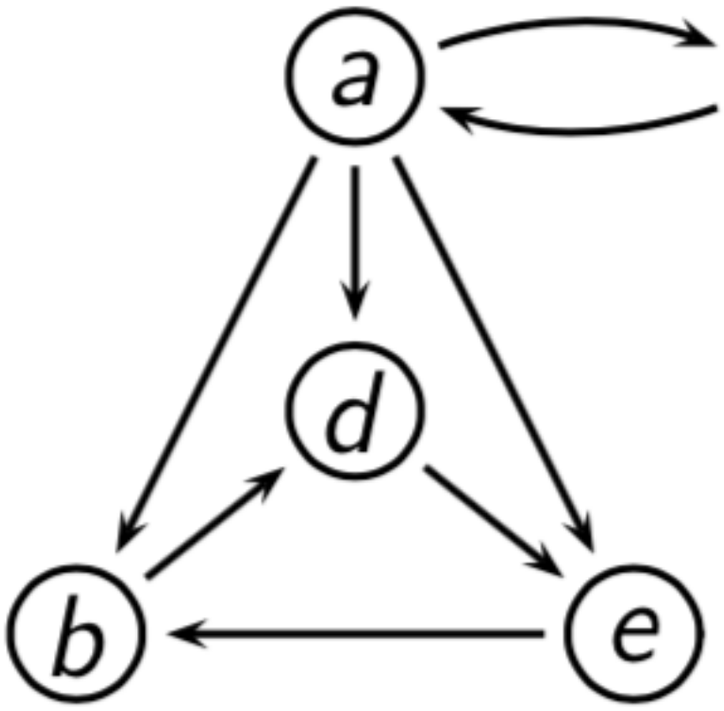


	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	0	0	0	1	0	0	0
c	1	0	0	0	0	0	0
d	0	0	0	0	1	0	0
e	0	1	0	0	0	0	0
f	0	0	0	0	0	0	0
g	0	0	0	0	0	1	0

Adjacency Matrix

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations: Directed Graphs





a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow d$
c	$\rightarrow a$
d	$\rightarrow e$
e	$\rightarrow b$
f	
g	$\rightarrow f$

Adjacency List

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

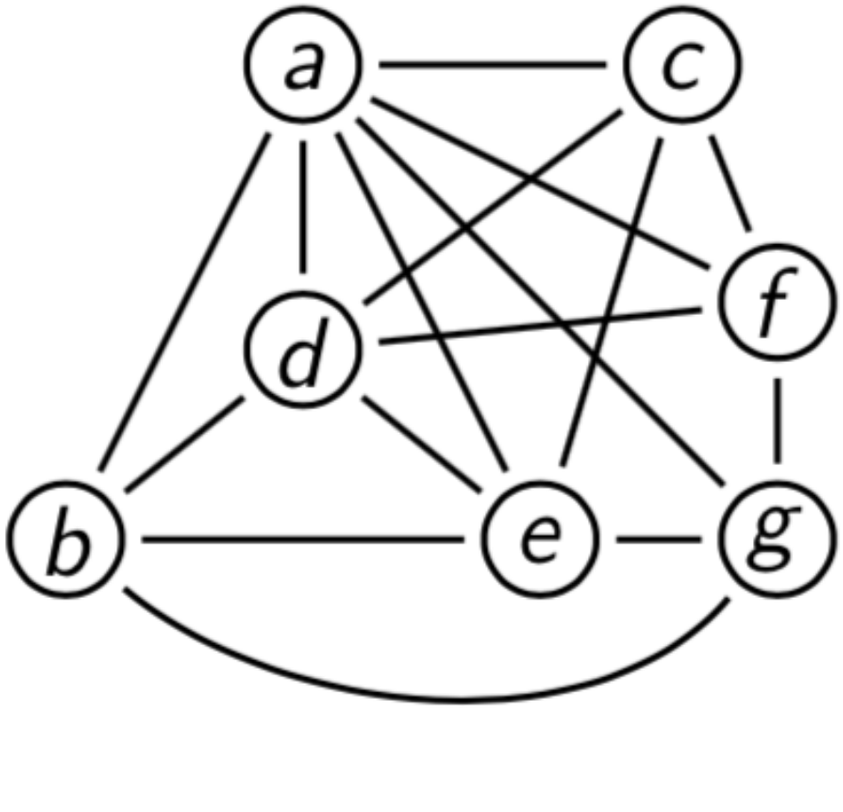
Left blank

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 1

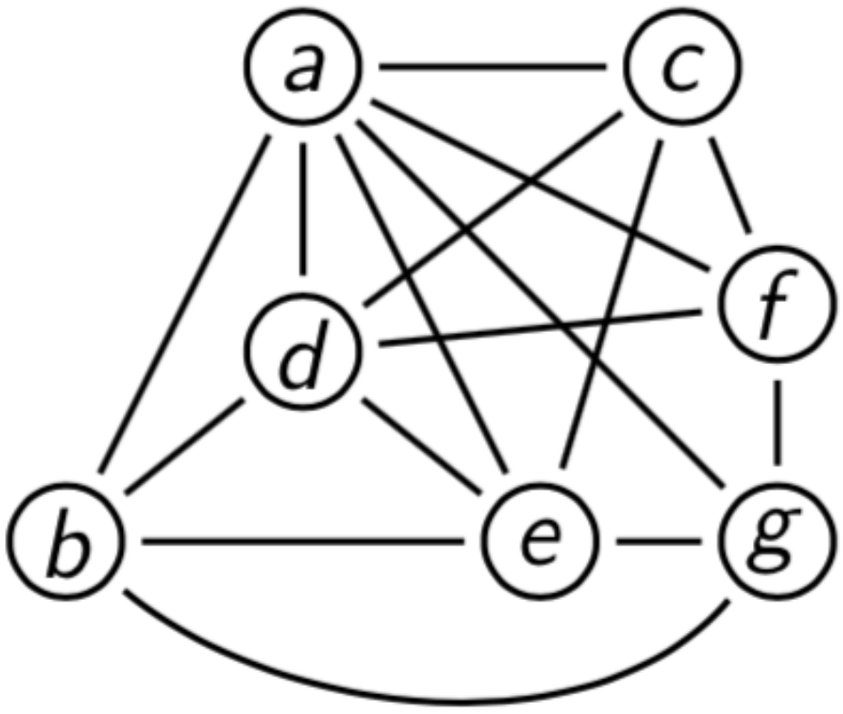
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 1



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

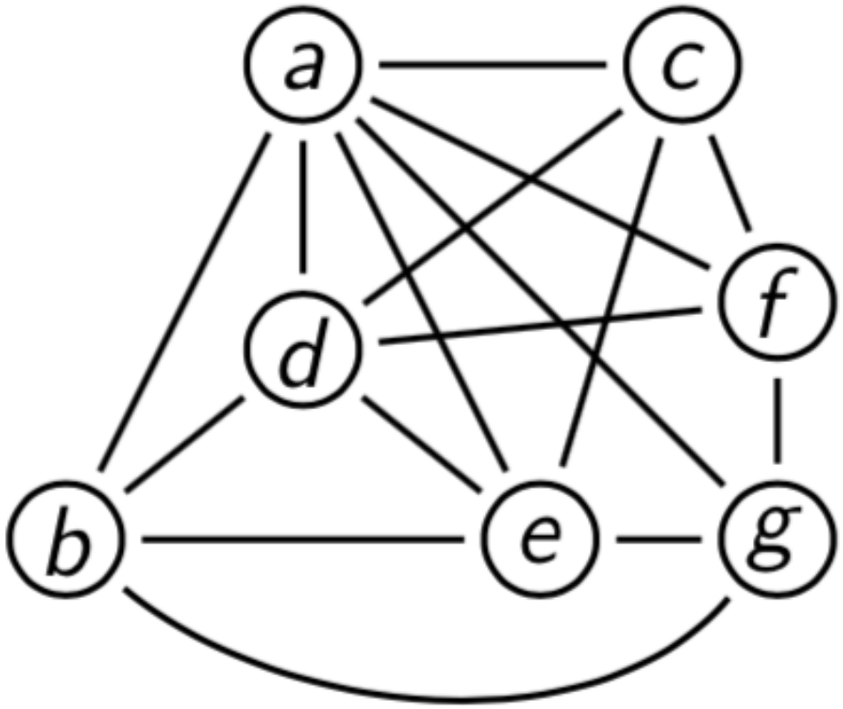
Question 1

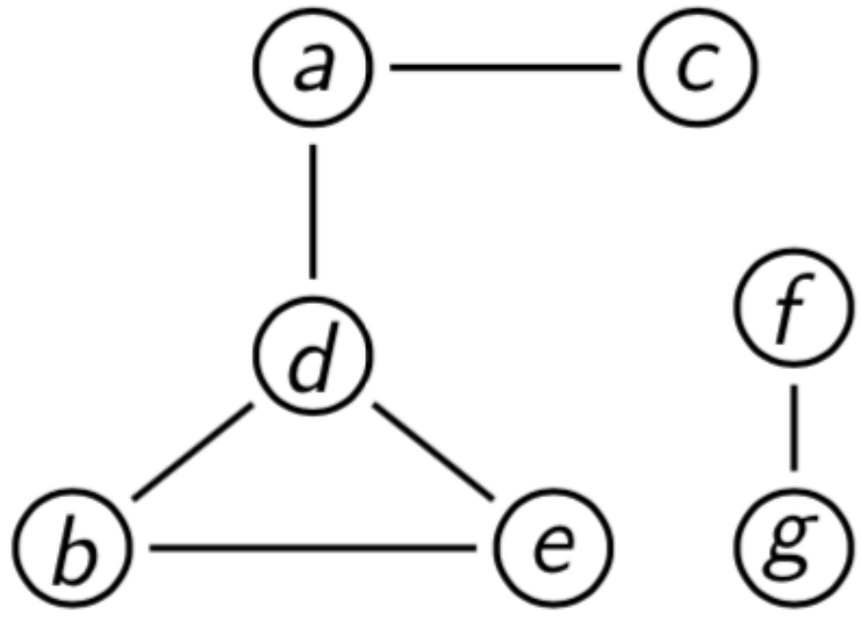


Dense graph

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 1

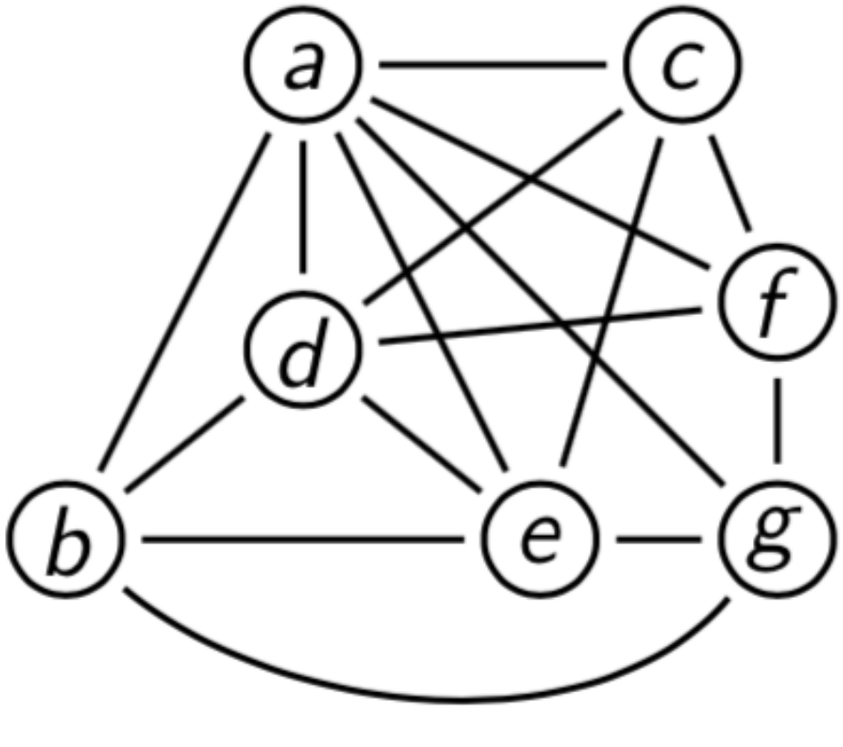




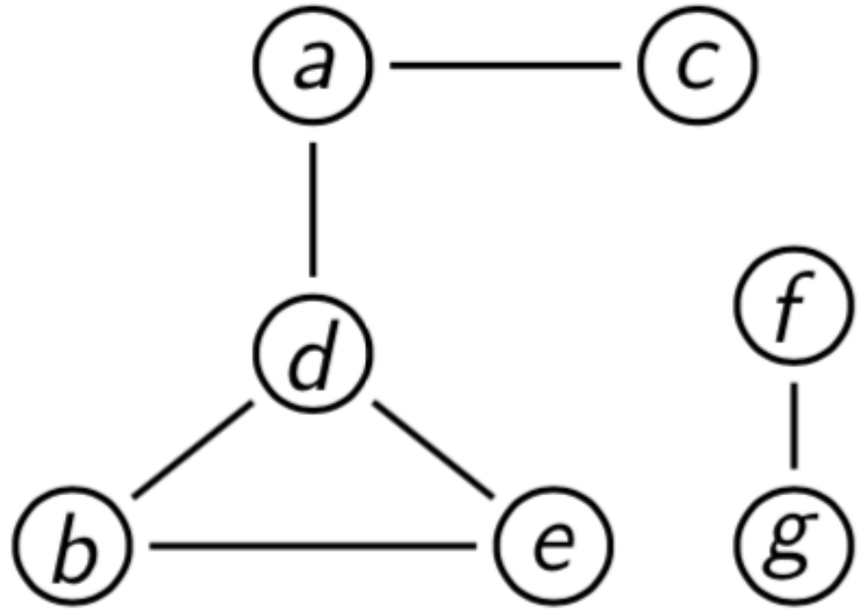
Dense graph

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 1



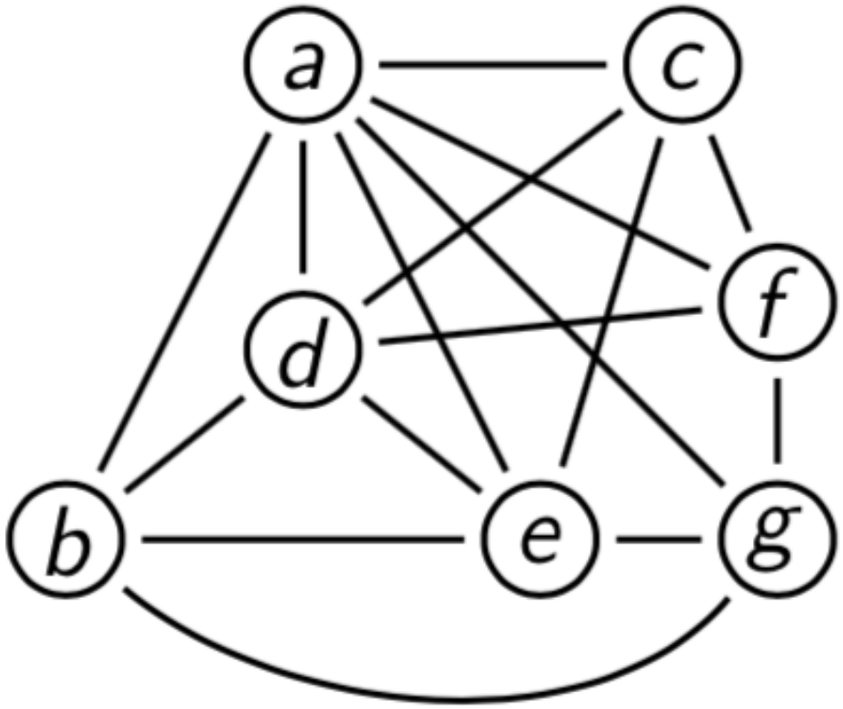
Dense graph



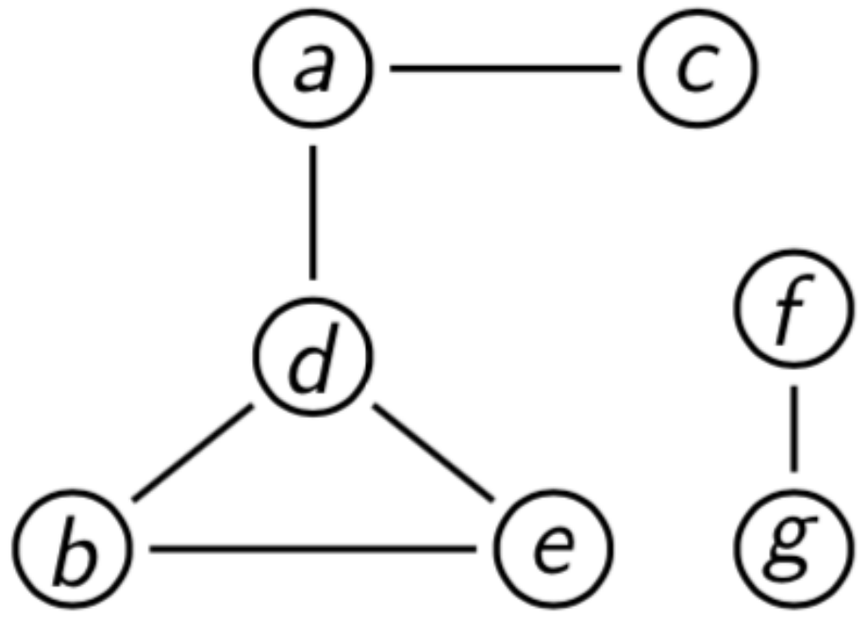
Sparse graph

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 1



Dense graph

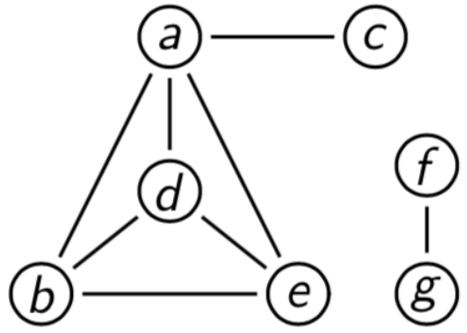


Sparse graph

Which representation (a matrix or a list) is more space efficient for each graph?

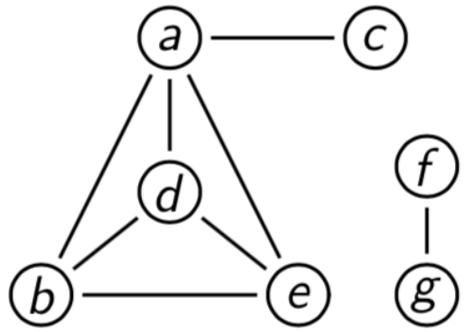
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

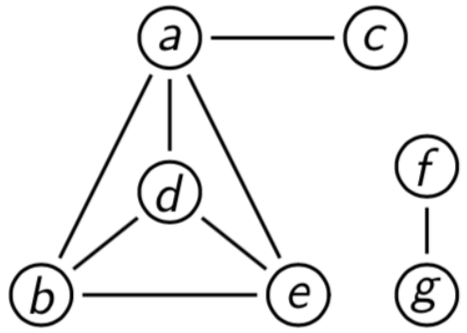
	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

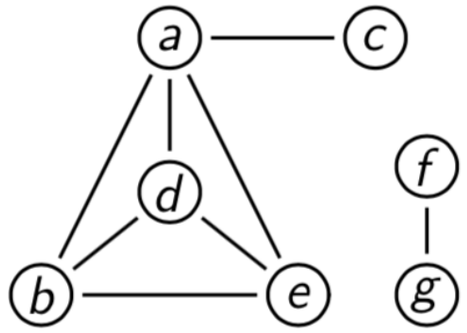
Adjacency List

a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

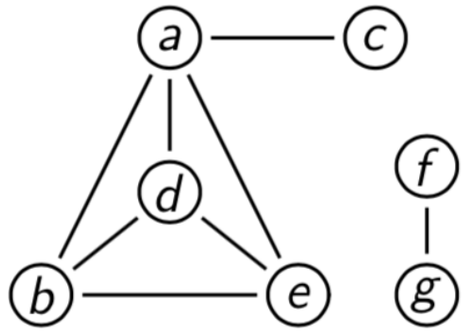
a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

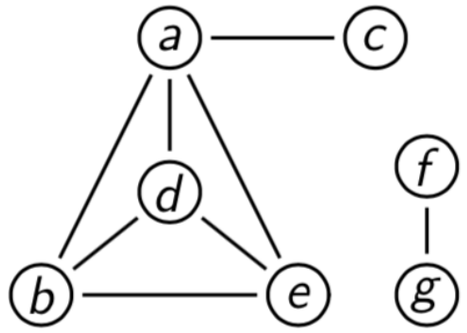
a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

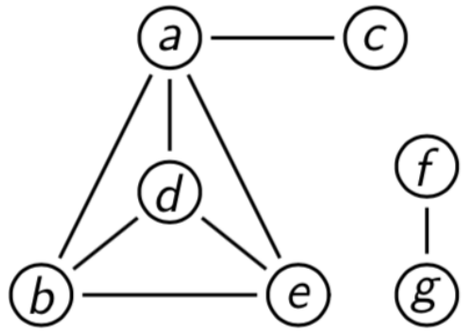
a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

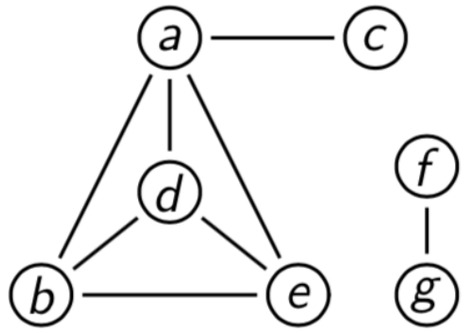
a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

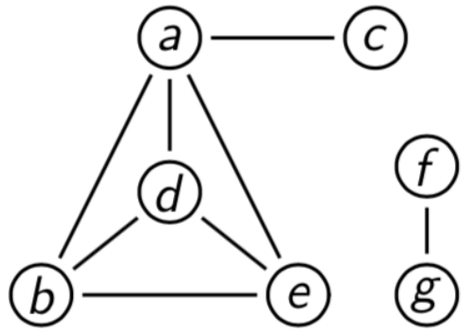
a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Question 2



Adjacency Matrix

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	1	0	0
c	1	0	0	0	0	0	0
d	1	1	0	0	1	0	0
e	1	1	0	1	0	0	0
f	0	0	0	0	0	0	1
g	0	0	0	0	0	1	0

Adjacency List

a	$\rightarrow b \rightarrow c \rightarrow d \rightarrow e$
b	$\rightarrow a \rightarrow d \rightarrow e$
c	$\rightarrow a$
d	$\rightarrow a \rightarrow b \rightarrow e$
e	$\rightarrow a \rightarrow b \rightarrow d$
f	$\rightarrow g$
g	$\rightarrow f$

Which graph representation would be better (more efficient) to answer a question:

is edge (v1,v2) in the graph?

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

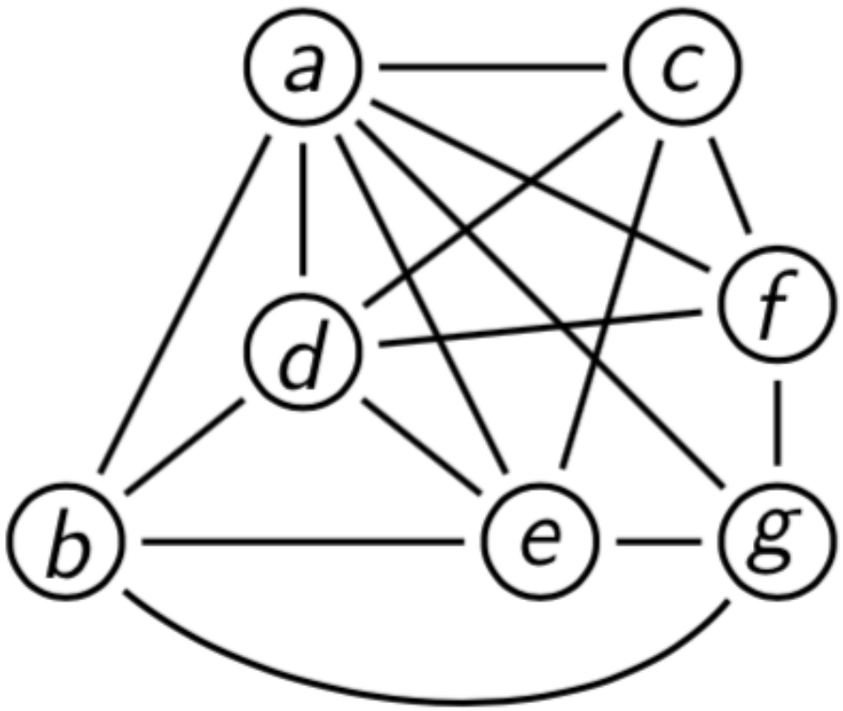
Graph Representations

Different kinds of representations are better suited to different kinds of graphs.

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations

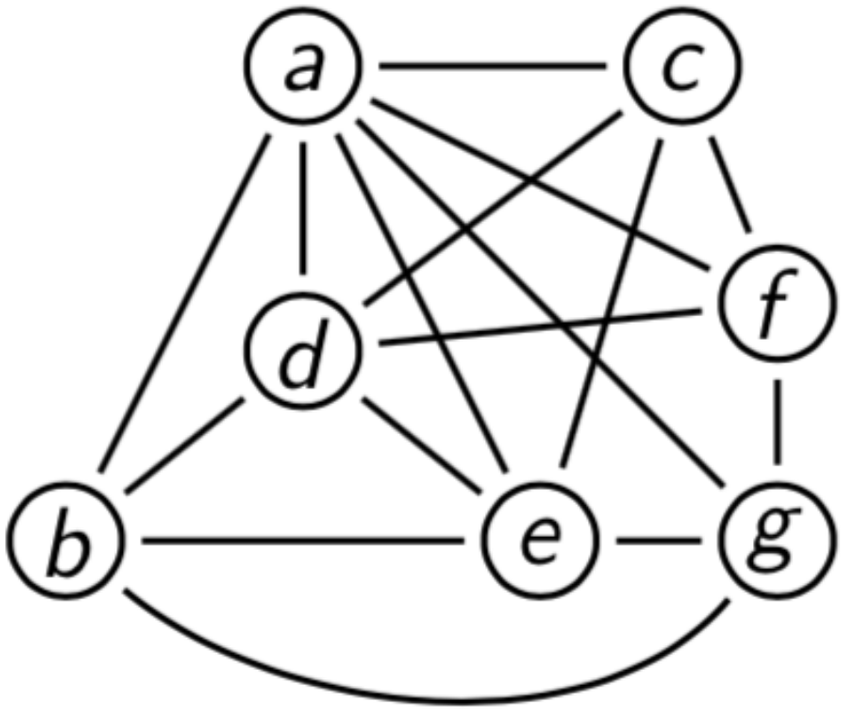
Different kinds of representations are better suited to different kinds of graphs.

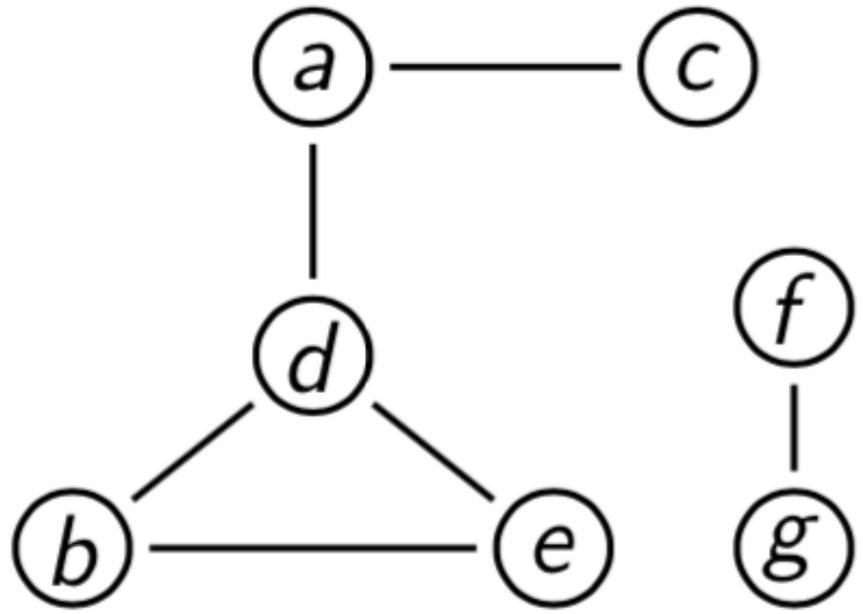


Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations

Different kinds of representations are better suited to different kinds of graphs.

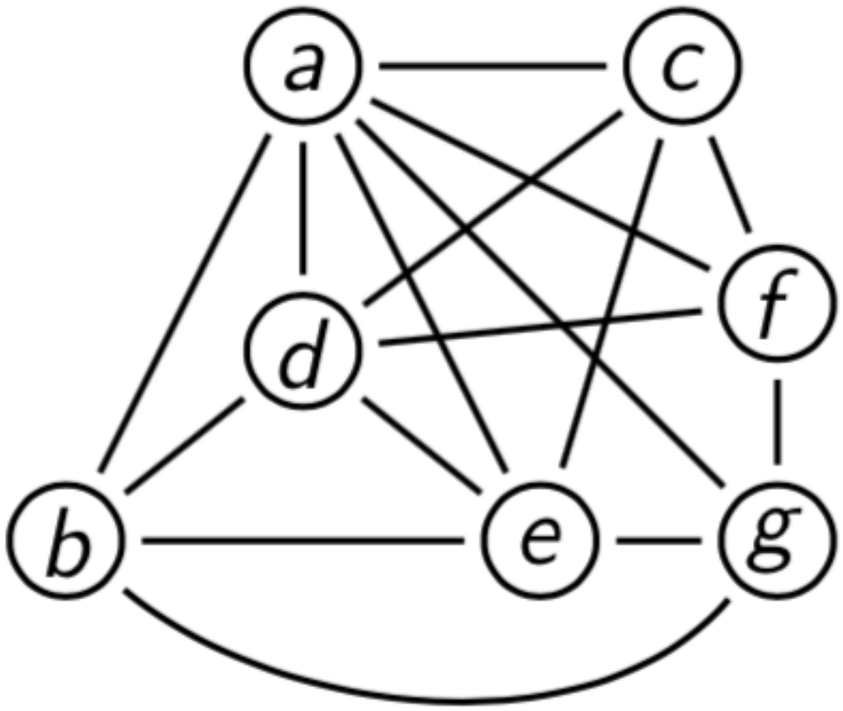




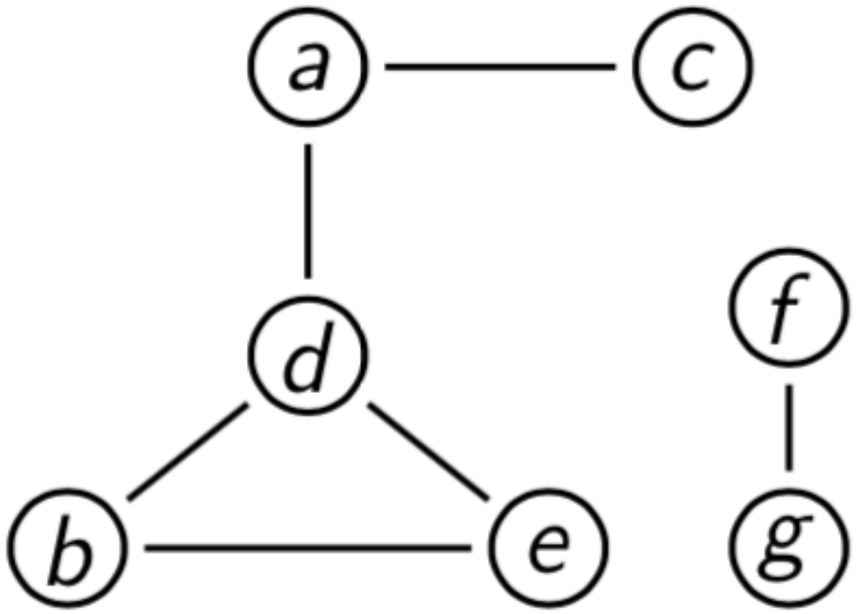
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations

Different kinds of representations are better suited to different kinds of graphs.



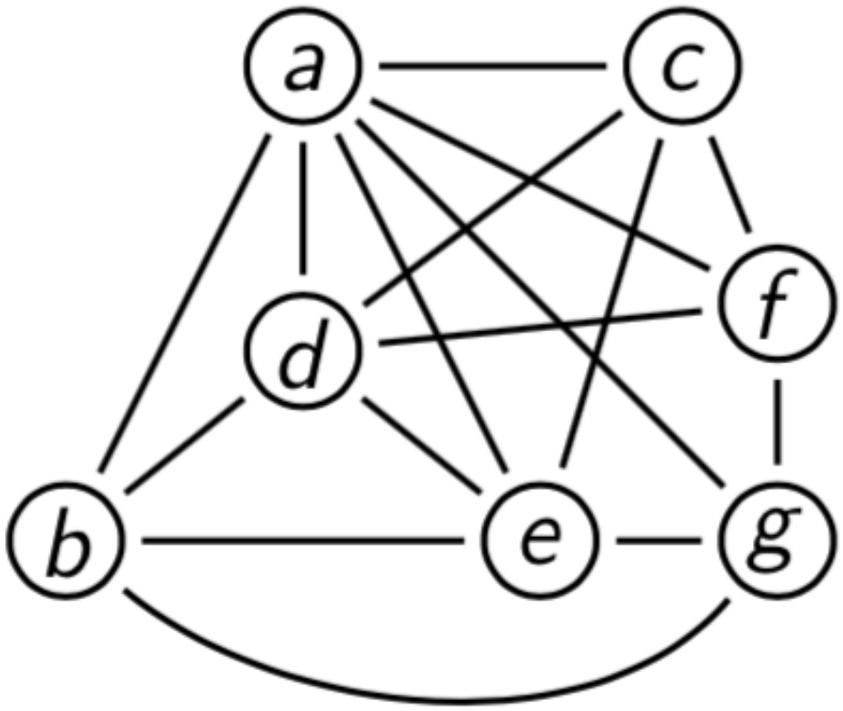
For a **dense graph** adjacency matrix might be better



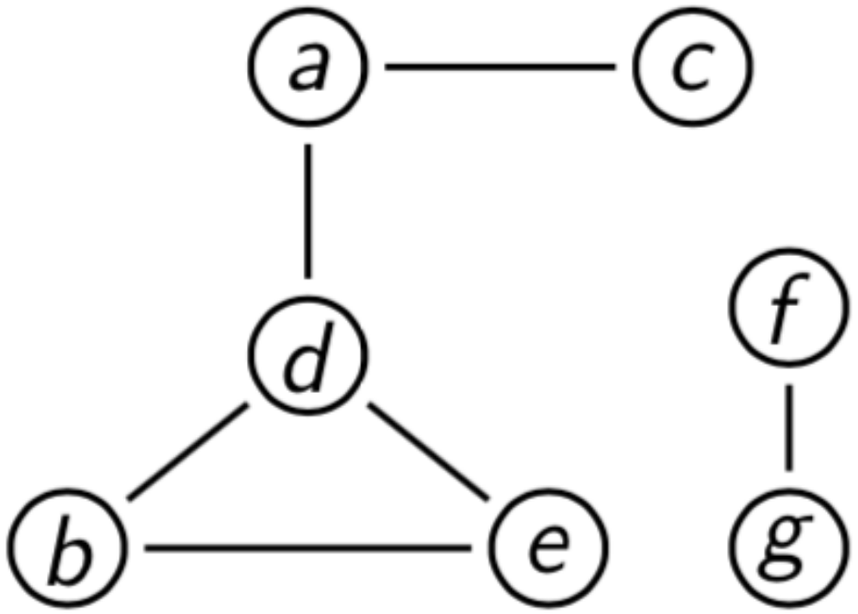
Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Graph Representations

Different kinds of representations are better suited to different kinds of graphs.



For a **dense graph** adjacency matrix might be better



For a **sparse graph** adjacency list might be better

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License

Next lecture

- Graph traversal, where we get down to the details of graph algorithms

Copyright University of Melbourne 2026, provided under Creative Commons Attribution License